CHAPTER

Data Types

CHAPTER 8

Every program uses data, either explicitly or implicitly, to arrive at a result. Data in a program are collected into data structures, which are manipulated by control structures that represent an algorithm. This is clearly expressed by the following pseudoequation (an equation that isn't mathematically exact but expresses the underlying principles),

$$\text{algorithms} = \text{data structures} + \text{programs}$$

which comes from the title of a book by Niklaus Wirth (Wirth [1976]). How a programming language expresses data and control largely determines how programs are written in the language. The present chapter studies the concept of data type as the basic concept underlying the representation of data in programming languages, while the chapters that follow study control.

Data in its most primitive form inside a computer is just a collection of bits. A language designer could take this view and build up all data from it. In essence, the designer could create a virtual machine (that is, a simulation of hardware—though not necessarily the actual hardware being used) as part of the definition of the language. However, such a language would not provide the kinds of abstraction necessary for large or even moderately sized programs. Thus, most programming languages include a set of simple data entities, such as integers, reals, and Booleans, as well as mechanisms for constructing new data entities from these. Such abstractions are an essential mechanism in programming languages and contribute to achieving the design goals of readability, writability, reliability, and machine independence. However, we should also be aware of the pitfalls to which such abstraction can lead.

One potential problem is that machine dependencies are often part of the implementation of these abstractions, and the language definition may not address these because they are hidden in the basic abstractions. An example of this is the **finitude** of all data in a computer, which is masked by the abstractions. For example, when we speak of integer data we often think of integers in the mathematical sense as an infinite set: $\ldots, -2, -1, 0, 1, 2, \ldots$, but in a computer's hardware there is always a largest and smallest integer. Sometimes a programming language's definition ignores this fact, thus making the language machine dependent.[1]

A similar situation arises with the precision of real numbers and the behavior of real arithmetic operations. This is a difficult problem for the language designer to address, since simple data types and arithmetic are usually built into hardware. A positive development was the establishment in 1985 by the Institute of Electrical and Electronics Engineers (IEEE) of a floating-point standard that attempts

[1]In a few languages, particularly functional languages, integers can be arbitrarily large and integer arithmetic is implemented in software; while this removes machine dependencies, it usually means arithmetic is too slow for computationally intensive algorithms.

to reduce the dependency of real number operations on the hardware (and encourages hardware manufacturers to ensure that their processors comply with the standard). C++, Ada, and Java all rely on some form of this standard to make floating-point operations more machine-independent. Indeed, in an attempt to reduce machine dependencies to a minimum, the designers of Java and Ada built into their standards strong requirements for all arithmetic operations. C++, too, has certain minimal requirements as part of its standard, but it is less strict than Ada and Java.

An even more significant issue regarding data types is the disagreement among language designers on the extent to which type information should be made explicit in a programming language and be used to verify program correctness prior to execution. Typically, the line of demarcation is clearest between those who emphasize maximal flexibility in the use of data types and who advocate no explicit typing or translation-time type verification, and those who emphasize maximum restrictiveness and call for strict implementation of translation-time type checking. A good example of a language with no explicit types or translation-time typing is Scheme,[2] while Ada is a good example of a very strictly type-checked language (sometimes referred to as a **strongly typed** language). There are many reasons to have some form of static (i.e., translation-time) type checking, however:

1. Static type information allows compilers to allocate memory efficiently and generate machine code that efficiently manipulates data, so **execution efficiency** is enhanced.
2. Static types can be used by a compiler to reduce the amount of code that needs to be compiled (particularly in a recompilation), thus improving **translation efficiency**.
3. Static type checking allows many standard programming errors to be caught early, improving **writability**, or efficiency of program coding.
4. Static type checking improves the **security** and **reliability** of a program by reducing the number of execution errors that can occur.
5. Explicit types in a programming language enhance **readability** by documenting data design, allowing programmers to understand the role of each data structure in a program, and what can and cannot be done with data items.
6. Explicit types can be used to **remove ambiguities** in programs. A prime example of this was discussed in the previous chapter (Section 7.4): Type information can be used to resolve overloading.
7. Explicit types combined with static type checking can be used by programmers as a **design tool**, so that incorrect design decisions show up as translation-time errors.
8. Finally, static typing of interfaces in large programs enhances the development of large programs by verifying **interface consistency** and **correctness**.

[2]To say that Scheme has no explicit types and no translation-time type checking is not to say that Scheme has no types. Indeed, every data value in Scheme has a type, and extensive checking is performed on the type of each value during execution.

Many language designers considered these arguments so compelling that they developed languages in which virtually every programming step required the exact specification in the source code of the data types in use and their properties. This led programmers and other language designers to complain (with some justification) that type systems were being used as mechanisms to force programmers into a rigid discipline that was actually *reducing* writability and good program design.[3]

Since these arguments were first made in the 1960s and 1970s, many advances have been made in making static typing more flexible, while at the same time preserving all or most of the previously listed advantages to static typing. Thus, most modern languages (except for special-purpose languages such as scripting and query languages) use some form of static typing, while using techniques that allow for a great deal of flexibility.

In this chapter, we first describe the primary notion of **data type** (the basic abstraction mechanism, common to virtually all languages), and the principal types and type constructors available in most languages. After discussing these basic principles of data types, we will describe the mechanisms of static type checking and type inference, and discuss some of the modern methods for providing flexibility in a type system, while still promoting correctness and security. Other mechanisms that provide additional flexibility and security come from modules, and will be studied in later chapters.

8.1 Data Types and Type Information

Program data can be classified according to their **types**. At the most basic level, virtually every data value expressible in a programming language has an implicit type. For example, in C the value 21 is of type `int`, 3.14159 is of type `double`, and `"hello"` is of type "array of `char`".[4] In Chapters 6 and 7, you saw that the types of variables are often explicitly associated with variables by a declaration, such as:

```
int x;
```

which assigns data type `int` to the variable `x`. In a declaration like this, a type is just a name (in this case the keyword `int`), which carries with it some properties, such as the kinds of values that can be stored, and the way these values are represented internally.

Some declarations implicitly associate a type with a name, such as the Pascal declaration

```
const PI = 3.14159;
```

which implicitly gives the constant `PI` the data type `real`, or the ML declaration

```
val x = 2;
```

which implicitly gives the constant `x` the data type `int`.

Since the internal representation is a system-dependent feature, from the abstract point of view, we can consider a type name to represent the possible values that a variable of that type can hold. Even more

[3]Bjarne Stroustrup, for example (Stroustrup [1994], p. 20), has written that he found Pascal's type system to be a "straightjacket that caused more problems than it solved by forcing me to warp my designs to suit an implementation-oriented artifact."

[4]This type is not expressible directly using C keywords, though the C++ type `const char*` could be used. In reality, we should distinguish the type from its keyword (or words) in a language, but we shall continue to use the simple expedient of using keywords to denote types whenever possible.

abstractly, we can consider the type name to be essentially identical to the set of values it represents, and we can state the following:

DEFINITION:1. A ***data type*** is a set of values.

Thus, the declaration of `x` as an `int` says that the value of `x` must be in the set of integers as defined by the language (and the implementation), or, in mathematical terminology (using $\in$ for membership) the declaration

```
int x;
```

means the same as:

$$\text{value of } x \in \textit{Integers}$$

where *Integers* is the set of integers as defined by the language and implementation. In Java, for example, this set is always the set of integers from to 22,147,483,648 to 2,147,483,647, since integers are always stored in 32-bit two's-complement form.

A data type as a set can be specified in many ways: It can be explicitly listed or enumerated, it can be given as a subrange of otherwise known values, or it can be borrowed from mathematics, in which case the finiteness of the set in an actual implementation may be left vague or ignored.[5] Set operations can also be applied to get new sets out of old (see below).

A set of values generally also has a group of operations that can be applied to the values. These operations are often not mentioned explicitly with the type, but are part of its definition. Examples include the arithmetic operations on integers or reals, the subscript operation `[ ]` on arrays, and the member selector operator on structure or record types. These operations also have specific properties that may or may not be explicitly stated [e.g., `(x + 1) - 1 = x` or `x + y = y + x`]. Thus, to be even more precise, we revise our first definition to include explicitly the operations, as in the following definition:

DEFINITION:2. A ***data type*** is a set of values, together with a set of operations on those values having certain properties.

In this sense, a data type is actually a mathematical algebra, but we will not pursue this view to any great extent here. (For a brief look at algebras, see the mathematics of abstract data types in Chapter 11.)

As we have already noted, a language translator can make use of the set of algebraic properties in a number of ways to assure that data and operations are used correctly in a program. For example, given a statement such as

```
z = x / y;
```

a translator can determine whether `x` and `y` have the same (or related) types, and if that type has a division operator defined for its values (thus also specifying which division operator is meant, resolving any

[5]In C, these limits are defined in the standard header files `limits.h` and `float.h`. In Java, the classes associated with each of the basic data types record these limits (for example, `java.lang.Integer.MAX_VALUE` 5 2147483647).

overloading). For example, in C and Java, if x and y have type int, then integer division is meant (with the remainder thrown away) and the result of the division also has type int, and if x and y have type double, then floating-point division is inferred.[6] Similarly, a translator can determine if the data type of z is appropriate to have the result of the division copied into it. In C, the type of z can be any numeric type (and any truncation is automatically applied), while in Java, z can only be a numeric type that can hold the entire result (without loss of precision).

The process a translator goes through to determine whether the type information in a program is consistent is called **type checking**. In the preceding example, type checking verifies that variables x, y, and z are used correctly in the given statement.

Type checking also uses rules for inferring types of language constructs from the available type information. In the example above, a type must be attached to the expression x / y so that its type may be compared to that of z (in fact, x / y may have type int or double, depending on the types of x and y). The process of attaching types to such expressions is called **type inference**. Type inference may be viewed as a separate operation performed during type checking, or it may be considered to be a part of type checking itself.

Given a group of basic types like int, double, and char, every language offers a number of ways to construct more complex types out of the basic types; these mechanisms are called **type constructors**, and the types created are called **user-defined types**. For example, one of the most common type constructors is the **array**, and the definition

```
int a[10];
```

creates in C (or C++) a variable whose type is "array of int" (there is no actual array keyword in C), and whose size is specified to be 10. This same declaration in Ada appears as:

```
a: array (1..10) of integer;
```

Thus, the "array" constructor takes a base type (int or integer in the above example), and a size or range indication, and constructs a new data type. This construction can be interpreted as implicitly constructing a new set of values, which can be described mathematically, giving insight into the way the values can be manipulated and represented.

New types created with type constructors do not automatically get names. Names are, however, extremely important, not only to document the use of new types but also for type checking (as we will describe in Section 8.6), and for the creation of recursive types (Section 8.3.5). Names for new types are created using a **type declaration** (called a **type definition** in some languages). For example, the variable a, created above as an array of 10 integers, has a type that has no name (an **anonymous type**). To give this type a name, we use a typedef in C:

```
typedef int Array_of_ten_integers[10];
Array_of_ten_integers a;
```

or a type declaration in Ada:

```
type Array_of_ten_integers is array (1..10) of integer;
a: Array_of_ten_integers;
```

[6]Ada, on the other hand, has different symbols for these two operations: div is integer division, while / is reserved for floating-point division.

With the definition of new user-defined types comes a new problem. During type checking a translator must often compare two types to determine if they are the same, even though they may be user-defined types coming from different declarations (possibly anonymous, or with different names). Each language with type declarations has rules for doing this, called **type equivalence** algorithms. The methods used for constructing types, the type equivalence algorithm, and the type inference and type correctness rules are collectively referred to as a **type system**.

If a programming language definition specifies a complete type system that can be applied statically and that guarantees that all (unintentional) data-corrupting errors in a program will be detected at the earliest possible point, then the language is said to be **strongly typed**. Essentially, this means that all type errors are detected at translation time, with the exception of a few errors that can only be checked during execution (such as array subscript bounds). In these cases, code is introduced to produce a runtime error. Strong typing ensures that most **unsafe programs** (i.e., programs with data-corrupting errors) will be rejected at translation time, and those unsafe programs that are not rejected at translation time will cause an execution error prior to any data-corrupting actions. Thus, no unsafe program can cause data errors in a strongly typed language. Unfortunately, strongly typed languages often reject safe programs as well as unsafe programs, because of the strict nature of their type rules. (That is, the set of **legal programs**—those accepted by a translator—is a *proper* subset of the set of safe programs.) Strongly typed languages also place an additional burden on the programmer in that appropriate type information must generally be explicitly provided, in order for type checking to work properly. The main challenge in the design of a type system is to minimize both the number of illegal safe programs and the amount of extra type information that the programmer must supply, while still retaining the property that all unsafe programs are illegal.

Ada is a strongly typed language that, nevertheless, has a fairly rigid type system, thus placing a considerable burden on the programmer. ML and Haskell are also strongly typed, but they place less of a burden on the programmer, and with fewer illegal but safe programs. Indeed, ML has a completely formalized type system in which all properties of legal programs are mathematically provable. Pascal and its related languages (such as Modula-2) are usually also considered to be strongly typed, even though they include a few loopholes. C has even more loopholes and so is sometimes called a **weakly typed language**. C++ has attempted to close some of the most serious type of loopholes of C, but for compatibility reasons it still is not completely strongly typed.

Languages without static type systems are usually called **untyped languages** (or **dynamically typed languages**). Such languages include Scheme and other dialects of Lisp, Smalltalk, and most scripting languages such as Perl, Python, and Ruby. Note, however, that an untyped language does not necessarily allow programs to corrupt data—this just means that all safety checking is performed at execution time. For example, in Scheme *all* unsafe programs will generate runtime errors, and no safe programs are illegal. This is an enviable situation from the point of view of strong typing, but one that comes with a significant cost. (All type errors cause unpleasant runtime errors, and the code used to generate these errors causes slower execution times.)

In the next section, we study the basic types that are the building blocks of all other types in a language. In Section 8.3, we study basic type constructors and their corresponding set interpretations. In Section 8.4, we give an overview of typical type classifications and nomenclature. In Section 8.5, we study type equivalence algorithms, and in Section 8.6, type-checking rules. Methods for allowing the programmer to override type checking are examined in Section 8.7. Sections 8.8 and 8.9 give an overview of **polymorphism**, in which names may have multiple types, while still permitting static type checking. Section 8.10 concludes the chapter with a discussion of type analysis in a parser for TinyAda.

8.2 Simple Types

Algol-like languages (C, Ada, Pascal), including the object-oriented ones (C++, Java), all classify data types according to a relatively standard basic scheme, with minor variations. Unfortunately, the names used in different language definitions are often different, even though the concepts are the same. We will attempt to use a generic name scheme in this section and then point out differences among some of the foregoing languages in the next.

Every language comes with a set of **predefined types** from which all other types are constructed. These are generally specified using either keywords (such as `int` or `double` in C++ or Java) or predefined identifiers (such as `String` or `Process` in Java). Sometimes variations on basic types are also predefined, such as `short`, `long`, `long double`, `unsigned int`, and so forth that typically give special properties to numeric types.

Predefined types are primarily **simple types**: types that have no other structure than their inherent arithmetic or sequential structure. All the foregoing types except `String` and `Process` are simple. However, there are simple types that are not predefined: **enumerated types** and **subrange types** are also simple types.

Enumerated types are sets whose elements are named and listed explicitly. A typical example (in C) is

```
enum Color {Red, Green, Blue};
```

or (the equivalent in Ada):

```
type Color_Type is (Red, Green, Blue);
```

or (the equivalent in ML):

```
datatype Color_Type = Red | Green | Blue;
```

In Ada and ML, enumerations are defined in a type declaration, and are truly new types. In most languages (but not ML), enumerations are **ordered**, in that the order in which the values are listed is important. Also, most languages include a predefined **successor** and **predecessor** operation for any enumerated type. Finally, in most languages, no assumptions are made about how the listed values are represented internally, and the only possible value that can be printed is the value name itself. As a runnable example, the short program in Figure 8.1 demonstrates Ada's enumerated type mechanism, which comes complete with symbolic I/O capability so that the actual defined names can be printed. In particular, see line 6, which allows enumeration values to be printed.

```
(1)  with Text_IO; use Text_IO;
(2)  with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;

(3)  procedure Enum is
(4)    type Color_Type is (Red,Green,Blue);
(5)    -- define Color_IO so that Color_Type values can be -- printed
(6)    package Color_IO is new Enumeration_IO(Color_Type);
(7)    use Color_IO;
```

Figure 8.1 An Ada program demonstrating the use of an enumerated type *(continues)*

(continued)

```
(8)    x : Color_Type := Green;
(9)  begin
(10)   x := Color_Type'Succ(x); -- x is now Blue
(11)   x := Color_Type'Pred(x); -- x is now Green
(12)   put(x); -- prints GREEN
(13)   new_line;
(14) end Enum;
```

Figure 8.1 An Ada program demonstrating the use of an enumerated type

By contrast, in C an `enum` declaration defines a type "`enum . . .`," but the values are all taken to be names of integers and are automatically assigned the values 0, 1, and so forth, unless the user initializes the `enum` values to other integers; thus, the C `enum` mechanism is really just a shorthand for defining a series of related integer constants, except that a compiler could use less memory for an `enum` variable. The short program in Figure 8.2, similar to the earlier Ada program, shows the use of C `enums`. Note the fact that the programmer can also control the actual values in an `enum` (line 3 of Figure 8.2), even to the point of creating overlapping values.

```
(1)  #include <stdio.h>

(2)  enum Color {Red,Green,Blue};
(3)   enum NewColor {NewRed = 3, NewGreen = 2, NewBlue = 2};

(4)  main(){
(5)    enum Color x = Green; /* x is actually 1 */
(6)    enum NewColor y = NewBlue; /* y is actually 2 */
(7)    x++; /* x is now 2, or Blue */
(8)    y--; /* y is now 1 -- not even in the enum */
(9)    printf("%d\n",x); /* prints 2 */
(10)    printf("%d\n",y); /* prints 1 */
(11)   return 0;
(12) }
```

Figure 8.2 A C program demonstrating the use of an enumerated type, similar to Figure 8.1

Java omits the `enum` construction altogether.[7]

Subrange types are contiguous subsets of simple types specified by giving a least and greatest element, as in the following Ada declaration:

```
type Digit_Type is range 0..9;
```

The type `Digit_Type` is a completely new type that shares the values 0 through 9 with other integer types, and also retains arithmetic operations, too, insofar as they make sense. This is useful if we want

[7]Java does have an `Enumeration` interface, but it is a different concept.

to distinguish this type in a program from other integer types, and if we want to minimize storage and generate runtime checks to make sure the values are always in the correct range. Languages in the C family (C, C++, Java) do not have subrange types, since the same effect can be achieved manually by writing the appropriate-sized integer type (to save storage), and by writing explicit checks for the values, such as in the following Java example:

```
byte digit; // digit can contain −128..127
...
if (digit > 9 || digit < 0) throw new DigitException();
```

Subrange types are still useful, though, because they cause such code to be generated automatically, and because type checking need not assume that subranges are interchangeable with other integer types.

Typically subranges are defined by giving the first and last element from another type for which, like the integers, every value has a next element and a previous element. Such types are called **ordinal types** because of the discrete order that exists on the set. All numeric integer types in every language are ordinal types, and enumerations and subranges are also typically ordinal types. Ordinal types always have comparison operators (like < and >=), and often also have successor and predecessor operations (or ++ and -- operations). Not all types with comparison operators are ordinal types, however: real numbers are ordered (i.e., 3.98, 3.99), but there is no successor or predecessor operation. Thus, a subrange declaration such as:

```
type Unit_Interval is range 0.0..1.0;
```

is usually illegal in most languages. Ada is a significant exception.[8]

Allocation schemes for the simple types are usually conditioned by the underlying hardware, since the implementation of these types typically relies on hardware for efficiency. However, as noted previously, languages are calling more and more for standard representations such as the IEEE 754 standard, which also requires certain properties of the operators applied to floating-point types. Some languages, like Java, also require certain properties of the integer types. If the hardware does not conform, then the language implementor must supply code to force compliance. For example, here is a description of the Java requirements for integers, taken from Arnold, Gosling, and Holmes [2000], page 156:

Integer arithmetic is modular two's complement arithmetic—that is, if a value exceeds the range of its type (`int` or `long`), it is reduced modulo the range. So integer arithmetic never overflows or underflows, but only wraps.

Integer division truncates toward zero (7/2 is 3 and 27/2 is 23). For integer types, division and remainder obey the rule:

```
(x/y)*y + x%y == x
```

So 7%2 is 1 and 27%2 is 21. Dividing by zero or remainder by zero is invalid for integer arithmetic and throws an `ArithmeticException`.

Character arithmetic is integer arithmetic after the `char` is implicitly converted to `int`.

[8] In Ada, however, the number of digits of precision must also be specified, so we must write `type Unit_Interval is digits 8 range 0.0 .. 1.0;`.

8.3 Type Constructors

Since data types are sets, set operations can be used to construct new types out of existing ones. Such operations include Cartesian product, union, powerset, function set, and subset.

When applied to types, these set operations are called **type constructors**. In a programming language, all types are constructed out of the predefined types using type constructors. In the previous section, we have already seen a limited form of one of these constructors—the subset construction—in subrange types. There are also type constructors that do not correspond to mathematical set constructions, with the principal example being pointer types, which involve a notion of storage not present in mathematical sets. There are also some set operations that do not correspond to type constructors, such as intersection (for reasons to be explained). In this section, we will catalog and give examples of the common type constructors.

8.3.1 Cartesian Product

Given two sets U and V, we can form the Cartesian or cross product consisting of all ordered pairs of elements from U and V:

$$U \times V = \{(u, v) \mid u \text{ is in } U \text{ and } v \text{ is in } V\}$$

Cartesian products come with projection functions $p_1: U \times V \to U$ and $p_2: U \times V \to V$, where $p_1((u, v)) = u$ and $p_2((u, v)) = v$. This construction extends to more than two sets. Thus $U \times V \times W = \{(u, v, w) \mid u \text{ in } U, v \text{ in } V, w \text{ in } W\}$. There are as many projection functions as there are components.

In many languages the Cartesian product type constructor is available as the **record** or **structure construction**. For example, in C the declaration

```
struct IntCharReal{
   int i;
   char c;
   double r;
};
```

constructs the Cartesian product type `int` × `char` × `double`.

In Ada this same declaration appears as:

```
type IntCharReal is record
   i: integer;
   c: character;
   r: float;
end record;
```

However, there is a difference between a Cartesian product and a record structure: In a record structure, the components have names, whereas in a product they are referred to by position. The projections in a record structure are given by the **component selector operation** (or **structure member operation**): If `x` is a variable of type `IntCharReal`, then `x.i` is the projection of `x` to the integers. Some authors, therefore,

consider record structure types to be different from Cartesian product types. Indeed, most languages consider component names to be part of the type defined by a record structure. Thus,

```
struct IntCharReal{
   int j;
   char ch;
   double d;
};
```

can be considered different from the `struct` previously defined, even though they represent the same Cartesian product set.

Some languages have a purer form of record structure type that is essentially identical to the Cartesian product, where they are often called **tuples**. For example, in ML, we can define `IntCharReal` as:

```
type IntCharReal = int * char * real;
```

Note that here the asterisk takes the place of the ×, which is not a keyboard character. All values of this type are then written as tuples, such as `(2,#"a",3.14)` or `(42,#"z",1.1)`.[9] The projection functions are then written as `#1`, `#2`, and so forth (instead of the mathematical notation p_1, p_2, etc.), so that `#3 (2,#"a",3.14)` = 3.14.

A data type found in object-oriented languages that is related to structures is the **class**. Classes, however, include functions that act on the data in addition to the data components themselves, and these functions have special properties that were studied in Chapter 5 (such functions are called **member functions** or **methods**). In fact, in C++, a `struct` is really another kind of class, as was noted in that chapter. The `class` data type is closer to our second definition of data type, which includes functions that act on the data. We will explore how functions can be associated with data later in this chapter and in subsequent chapters.

8.3.2 Union

A second construction, the union of two types, is formed by taking the set theoretic union of their sets of values. Union types come in two varieties: discriminated unions and undiscriminated unions. A union is **discriminated** if a **tag** or **discriminator** is added to the union to distinguish which type the element is—that is, which set it came from. Discriminated unions are similar to disjoint unions in mathematics. Undiscriminated unions lack the tag, and assumptions must be made about the type of any particular value (in fact, the existence of undiscriminated unions in a language makes the type system *unsafe*, in the sense discussed in Section 8.2).

In C (and C++) the `union` type constructor constructs undiscriminated unions:

```
union IntOrReal{
   int i;
   double r;
};
```

Note that, as with `structs`, there are names for the different components (`i` and `r`). These are necessary because their use tells the translator which of the types the raw bits stored in the union should be

[9]Characters in ML use double quotation marks preceded by a # sign.

interpreted as; thus, if x is a variable of type `union IntOrReal`, then `x.i` is always interpreted as an `int`, and `x.r` is always interpreted as a `double`. These component names should not be confused with a discriminant, which is a separate component that indicates which data type the value *really* is, as opposed to which type we may think it is. A discriminant can be imitated in C as follows:

```
enum Disc {IsInt,IsReal};
struct IntOrReal{
   enum Disc which;
   union{
      int i;
      double r;
   } val;
};
```

and could be used as follows:

```
IntOrReal x;
x.which = IsReal;
x.val.r = 2.3;
...
if (x.which == IsInt) printf("%d\n",x.val.i);
else printf("%g\n",x.val.r);
```

Of course, this is still unsafe, since it is up to the programmer to generate the appropriate test code (and the components can also be assigned individually and arbitrarily).

Ada has a completely safe union mechanism, called a **variant record**. The above C code written in Ada would look as follows:

```
type Disc is (IsInt, IsReal);
type IntOrReal (which: Disc) is
record
   case which is
      when IsInt =>  i: integer;
      when IsReal => r: float;
   end case;
end record;
...
x: IntOrReal := (IsReal,2.3);
```

Note how in Ada the IntOrReal variable x must be assigned both the discriminant and a corresponding appropriate value at the same time—it is this feature that guarantees safety. Also, if the code tries to access the wrong variant during execution (which cannot be predicted at translation time):

```
put(x.i); -- but x.which = IsReal at this point
```

then a CONSTRAINT_ERROR is generated and the program halts execution.

Functional languages that are strongly typed also have a safe union mechanism that extends the syntax of enumerations to include data fields. Recall from Section 8.2 that ML declares an enumeration as in the following example (using vertical bar for "or"):

```
datatype Color_Type = Red | Green | Blue;
```

This syntax can be extended to get an IntOrReal union type as follows:

```
datatype IntOrReal = IsInt of int | IsReal of real;
```

Now the "tags" IsInt and IsReal are used directly as discriminants to determine the kind of value in each IntOrReal (rather than as member or field names). For example,

```
val x = IsReal(2.3);
```

creates a value with a real number (and stores it in constant x), and to access a value of type IntOrReal we must use code that tests which kind of value we have. For example, the following code for the printIntOrReal function uses a case expression and **pattern matching** (i.e., writing out a sample format for the translator, which then fills in the values if the actual value matches the format):

```
fun printInt x =
   (print("int: "); print(Int.toString x); print("\n"));

fun printReal x =
   (print("real: "); print(Real.toString x); print("\n"))

fun printIntOrReal x =
   case x of
      IsInt(i) => printInt i |
      IsReal(r) =>printReal r ;
```

The printIntOrReal function can now be used as follows:

```
printIntOrReal(x); (* prints "real: 2.3" *)
printIntOrReal (IsInt 42); (* prints "int: 42" *)
```

The tags IsInt and IsReal in ML are called **data constructors**, since they construct data of each kind within a union; they are similar to the object constructors in an object-oriented language. (Data constructors and the pattern matching that goes with them were studied in Chapter 3.)

Unions can be useful in reducing memory allocation requirements for structures when different data items are not needed simultaneously. This is because unions are generally allocated space equal to the maximum of the space needed for individual components, and these components are stored in overlapping regions of memory. Thus, a variable of type IntOrReal (without the discriminant) might be allocated 8 bytes. The first four would be used for integer variants, and all 8 would be used for a real variant. If a discriminant field is added, 9 bytes would be needed, although the number would be probably rounded to 10 or 12.

Unions, however, are not needed in object-oriented languages, since a better design is to use inheritance to represent different nonoverlapping data requirements (see Chapter 5). Thus, Java does not have unions. C++ does retain unions, presumably primarily for compatibility with C.

Even in C, however, unions cause a small but significant problem. Typically, unions are used as a variant part of a `struct`. This requires that an extra level of member selection must be used, as in the `struct IntOrReal` definition above, where we had to use `x.val.i` and `x.val.r` to address the overlapping data. C++ offers a new option here—the **anonymous union** within a `struct` declaration, and we could write `struct IntOrReal` in C++ as follows:

```
enum Disc {IsInt,IsReal};
struct IntOrReal{   // C++ only!
   enum Disc which;
   union{
      int i;
      double r;
   }; // no member name here
};
```

Now we can address the data simply as `x.i` and `x.r`. Of course, Ada already has this built into the syntax of the variant record.

8.3.3 Subset

In mathematics, a subset can be specified by giving a rule to distinguish its elements, such as pos_int = $\{x \mid x$ is an integer and $x > 0\}$. Similar rules can be given in programming languages to establish new types that are subsets of known types. Ada, for example, has a very general **subtype** mechanism. By specifying a lower and upper bound, subranges of ordinal types can be declared in Ada, as for example:

```
subtype IntDigit_Type is integer range 0..9;
```

Compare this with the simple type defined in Section 8.2:

```
type Digit_Type is range 0..9;
```

which is a completely new type, *not* a subtype of `integer`.

Variant parts of records can also be fixed using a subtype. For example, consider the Ada declaration of `IntOrReal` in the preceding section a subset type can be declared that fixes the variant part (which then must have the specified value):

```
subtype IRInt is IntOrReal(IsInt);
subtype IRReal is IntOrReal(IsReal);
```

Such subset types **inherit** operations from their parent types. Most languages, however, do not allow the programmer to specify which operations are inherited and which are not. Instead, operations are automatically or implicitly inherited. In Ada, for example, subtypes inherit all the operations of the parent type. It would be nice to be able to exclude operations that do not make sense for a subset type; for example, unary minus makes little sense for a value of type `IntDigit_Type`.

An alternative view of the subtype relation is to *define* it in terms of sharing operations. That is, a type S is a subtype of a type T if and only if all of the operations on values of type T can also be applied to values of type S. With this definition, a subset may not be a subtype, and a subtype may not be a subset. This view, however, is a little more abstract than the approach we take in this chapter.

Inheritance in object-oriented languages can also be viewed as a subtype mechanism, in the same sense of sharing operations as just described, with a great deal more control over which operations are inherited.

8.3.4 Arrays and Functions

The set of all functions $f: U \rightarrow V$ can give rise to a new type in two ways: as an **array type** or as a **function type**. When U is an ordinal type, the function f can be thought of as an **array** with **index type** U and **component type** V: if i is in U, then $f(i)$ is the ith component of the array, and the whole function can be represented by the sequence or tuple of its values $(f(low), \ldots, f(high))$, where *low* is the smallest element in U and *high* is the largest. (For this reason, array types are sometimes referred to as **sequence types**.) In C, C++, and Java the index set is always a positive integer range beginning at zero, while in Ada any ordinal type can be used as an index set.

Typically, array types can be defined either with or without sizes, but to define a *variable* of an array type it's typically necessary to specify its size at translation time, since arrays are normally allocated statically or on the stack (and thus a translator needs to know the size).[10] For example, in C we can define array types as follows:

```
typedef int TenIntArray [10];
typedef int IntArray [];
```

and we can now define variables as follows:

```
TenIntArray x;
int y[5];
int z[] = {1,2,3,4};
IntArray w = {1,2};
```

Note that each of these variables has its size determined statically, either by the array type itself or by the initial value list: `x` has size 10; `y`, 5; `z`, 4; and `w`, 2. Also, it is illegal to define a variable of type `IntArray` without giving an initial value list:

```
IntArray w; /* illegal C! */
```

Indeed, in C the size of an array cannot even be a computed constant—it must be a literal:

```
const int Size = 5;
int x[Size]; /* illegal C, ok in C++ (see Section 8.6.2) */
int x[Size*Size] /* illegal C, ok in C++ */
```

And, of course, any attempt to dynamically define an array size is illegal (in both C and C++):[11]

[10] Arrays can be allocated dynamically on the stack, but this complicates the runtime environment; see Chapter 10.
[11] This restriction has been removed in the 1999 C Standard.

```
int f(int size) {
   int a[size]; /* illegal */
   ...
}
```

C does allow arrays without specified size to be parameters to functions (these parameters are essentially pointers—see later in this section):

```
int array_max (int a[], int size){
   int temp, i;
   assert(size > 0);
   temp = a[0];
   for (i = 1; i < size; i++)
      if (a[i] > temp) temp = a[i];
   return temp;
}
```

Note in this code that the size of the array a had to be passed as an additional parameter. The size of an array is not part of the array (or of an array type) in C (or C++).

Java takes a rather different approach to arrays. Like C, Java array indexes must be nonnegative integers starting at 0. However, unlike C, Java arrays are always dynamically (heap) allocated. Also, unlike C, in Java the size of an array can be specified completely dynamically (but once specified cannot change unless reallocated). While the size is not part of the array type, the size *is* part of the information stored when an array is allocated (and is called its **length**). Arrays can also be defined using C-like syntax, or in an alternate form that associates the "array" property more directly with the component type. Figure 8.3 contains an example of some array declarations and array code in Java packaged into a runnable example.

```
import java.io.*;
import java.util.Scanner;

public class ArrayTest{

   static private int array_max(int[] a){     // note location of []
      int temp;
      temp = a[0];
      // length is part of a
      for (int i = 1; i < a.length; i++)
         if (a[i] > temp) temp = a[i];
      return temp;
   }
```

Figure 8.3 A Java program demonstrating the use of arrays *(continues)*

(continued)

```
    public static void main (String args[]){   // this placement of [] also
                                               // allowed
       System.out.print("Input a positive integer: ");
       Scanner reader = new Scanner(System.in));
       int size = reader.nextInt();
       int[] a = new int[size] ; // Dynamic array allocation
       for (int i = 0; i < a.length; i++) a[i] = i;
       System.out.println(array_max(a));
    }
 }
```

Figure 8.3 A Java program demonstrating the use of arrays

Ada, like C, allows array types to be declared without a size (so-called **unconstrained arrays**) but requires that a size be given when array variables (but not parameters) are declared. For example, the declaration

```
type IntToInt is array (integer range <>) of integer;
```

creates an array type from a subrange of integers to integers. The brackets "<>" indicate that the precise subrange is left unspecified. When a variable is declared, however, a range must be given:

```
table : IntToInt(-10..10);
```

Unlike C (but like Java), array ranges in Ada can be given dynamically. Unlike Java, Ada does not allocate arrays on the heap, however (see Chapter 10). Figure 8.4 represents a translation of the Java program of Figure 8.3, showing how arrays are used in Ada.

```
with Text_IO; use Text_IO;
with Ada.Integer_Text_IO;
use Ada.Integer_Text_IO;

procedure ArrTest is
   type IntToInt is array (INTEGER range <>) of INTEGER;

   function array_max(a: IntToInt) return integer is
      temp: integer;
      begin
         temp := a(a'first);     -- first subscript value for a
                                 -- a'range = set of legal subscripts
         for i in a'range loop
            if a(i) > temp then
```

Figure 8.4 An Ada program demonstrating the use of arrays, equivalent to the Java program of Figure 8.3 *(continues)*

(continued)

```
                temp := a(i);
             end if;
          end loop;
          return temp;
       end array_max;

    size: integer;

    begin
       put_line("Input a positive integer: ");
       get(size);
       declare
          x: IntToInt(1..size);      -- dynamically sized array
          max: integer;
          begin
             for i in x'range loop   -- x'range = 1..size
                x(i) := i;
             end loop;
             put(array_max(x));
             new_line;
          end;
    end ArrTest;
```

Figure 8.4 An Ada program demonstrating the use of arrays, equivalent to the Java program of Figure 8.3

Multidimensional arrays are also possible, with C/C++ and Java allowing arrays of arrays to be declared:

```
int x[10][20]; /* C code */
int[][] x = new int [10][20]; // Java code
```

Ada and Fortran have separate multidimensional array constructs as in the following Ada declaration:

```
type Matrix_Type is
   array (1..10,−10..10) of integer;
```

In Ada this type is viewed as different from the type:

```
type Matrix_Type is
   array (1..10) of array (−10..10) of integer;
```

Variables of the first type must be subscripted as `x(i,j)`, while variables of the second type must be subscripted as in `x(i)(j)`. (Note that square brackets are never used in Ada.)

Arrays are perhaps the most widely used type constructor, since their implementation can be made extremely efficient. Space is allocated sequentially in memory, and indexing is performed by an offset

calculation from the starting address of the array. In the case of multidimensional arrays, allocation is still linear, and a decision must be made about which index to use first in the allocation scheme. If `x` is defined (in Ada) as follows:

```
x: array (1..10,-10..10) of integer;
```

then `x` can be stored as `x[1,-10], x[1,-9], x[1,-8], . . . , x[1,10], x[2,-10], x[2,-9]`, and so on (**row-major form**) or as `x[1,-10], x[2,-10], x[3,-10], . . . , x[10,-10], x[1,-9], x[2,-9]`, and so on (**column-major form**). Note that if the indices can be supplied separately (from left to right), as in the C declaration

```
int x[10][20];
```

then row-major form *must* be used. Only if all indices must be specified together can column-major form be used (in FORTRAN this is the case, and column-major form is generally used).

Multidimensional arrays have a further quirk in C/C++ because of the fact that the size of the array is not part of the array and is not passed to functions (see the previous `array_max` function in C). If a parameter is a multidimensional array, then the size of all the dimensions except the first must be specified in the parameter declaration:

```
int array_max (int a[][20] , int size)
     /* size of second dimension required ! */
```

The reason is that, using row-major form, the compiler must be able to compute the distance in memory between `a[i]` and `a[i+1]`, and this is only possible if the size of subsequent dimensions is known at compile time. This is not an issue in Java or Ada, since the size of an array is carried with an array value dynamically.

Functional languages usually do not supply an array type, since arrays are specifically designed for imperative rather than functional programming. Some functional languages have imperative constructs, though, and those that do often supply some sort of array mechanism. Scheme has, for example, a **vector** type, and some versions of ML have an array module. Typically, functional languages use the **list** in place of the array, as discussed in Chapter 3.

General function and procedure types can also be created in some languages. For example, in C we can define a function type from integers to integers as follows:

```
typedef int (*IntFunction)(int);
```

and we can use this type to define either variables or parameters:

```
int square(int x) { return x*x; }
IntFunction f = square;
int evaluate(IntFunction g, int value)
{   return g(value); }
...
printf("%d\n", evaluate(f,3)); /* prints 9 */
```

Note that, as we remarked in the previous chapter, C requires that we define function variables, types, and parameters using pointer notation, but then does not require that we perform any dereferencing (this is to distinguish function definitions from function types).

Of course, functional languages have very general mechanisms of this kind as well. For example, in ML we can define a function type, such as the one in the previous code segment, in the following form:

```
type IntFunction = int -> int;
```

and we can use it in a similar way to the previous C code:

```
fun square (x: int) = x * x;
val f = square;
fun evaluate (g: IntFunction, value: int) = g value;
...
evaluate(f,3); (* evaluates to 9 *)
```

In Ada95, we can also write function parameters and types. Here is the `evaluate` function example in Ada95:

```
type IntFunction is
          access function (x:integer) return integer;
-- "access" means pointer in Ada

function square (x: integer) return integer is
   begin
      return x * x;
   end square;

function evaluate (g: IntFunction; value: integer) return integer is
   begin
      return g(value);
   end evaluate;

f: IntFunction := square'access;
-- note use of access attribute to get pointer to square
...
evaluate(f,3); -- evaluates to 9
```

By contrast, most pure object-oriented languages, such as Java and Smalltalk, have no function variables or parameters. These languages focus on objects rather than functions.

The format and space allocation of function variables and parameters depend on the size of the address needed to point to the code representing the function and on the runtime environment required by the language. See Chapter 10 for more details.

8.3.5 Pointers and Recursive Types

A type constructor that does not correspond to a set operation is the **reference** or **pointer** constructor, which constructs the set of all addresses that refer to a specified type. In C the declaration

```
typedef int* IntPtr;
```

constructs the type of all addresses where integers are stored. If `x` is a variable of type `IntPtr`, then it can be **dereferenced** to obtain a value of type integer: `*x = 10` assigns the integer value 10 to the location given by `x`. Of course, `x` must have previously been assigned a valid address; this is usually accomplished dynamically by a call to an allocation function such as `malloc` in C. (Pointer variables were discussed in Chapter 7.)

The same declaration as the foregoing in Ada is:

```
type IntPtr is access integer;
```

Pointers are implicit in languages that perform automatic memory management. For example, in Java all objects are implicitly pointers that are allocated explicitly (using the `new` operator like C++) but deallocated automatically by a garbage collector. Functional languages like Scheme, ML, or Haskell also use implicit pointers for data but do both the allocation and deallocation automatically, so that there is no syntax for pointers (and no `new` operation).

Sometimes languages make a distinction between pointers and **references**, a reference being the address of an object under the control of the system, which cannot be used as a value or operated on in any way (although it may be copied), while a pointer can be used as a value and manipulated by the programmer (including possibly using arithmetic operators on it). In this sense, pointers in Java are actually references, because, except for copying and the `new` operation, they are under the control of the system. C++ is perhaps the only language where both pointers and references exist together, although in a somewhat uneasy relationship. Reference types are created in C++ by a **postfix & operator**, not to be confused with the *prefix* address-of & operator, which returns a pointer. For example:

```
double r = 2.3;
double& s = r; // s is a reference to r – C++ only!
               // so they share memory
s += 1; // r and s are now both 3.3
cout << r << endl; // prints 3.3
```

This can also be implemented in C++ (also valid in C) using pointers:

```
double r = 2.3;
double* p = &r; // p has value = address of r
*p += 1; // r is 3.3
cout << r << endl; // prints 3.3
```

References in C++ are essentially constant pointers that are dereferenced every time they are used (Stroustrup [1997], p. 98). For example, in the preceding code, `s += 1` is the same as `r += 1` (`s` is dereferenced first), while `p +=1` increments the *pointer* value of `p`, so that `p` is now pointing to a different location (actually 8 bytes higher, if a `double` occupies 8 bytes).

A further complicating factor in C++ and C is that arrays are implicitly constant pointers to their first component. Thus, we can write code in C or C++ as follows:

```
int a[] = {1,2,3,4,5}; /* a[0] = 1, etc. */
int* p = a; /* p points to first component of a */
printf("%d\n", *p); /* prints value of a[0], so 1 */
printf("%d\n", *(p+2)); /* prints value of a[2]= 3 */
printf("%d\n", *(a+2)); /* also prints 3 */
printf("%d\n", 2[a]); /* also prints 3! */
```

Indeed, `a[2]` in C is just a shorthand notation for `a + 2`, and, by the commutativity of addition, we can equivalently write `2[a]`!

Pointers are most useful in the creation of **recursive types**: a type that uses itself in its declaration. Recursive types are extremely important in data structures and algorithms, since they naturally correspond to recursive algorithms, and represent data whose size and structure is not known in advance, but may change as computation proceeds. Two typical examples are lists and binary trees. Consider the following C-like declaration of lists of characters:

```
struct CharList{
   char data;
   struct CharList next; /* not legal C! */
};
```

There is no reason in principle why such a recursive definition should be illegal; recursive functions have a similar structure. However, a close look at this declaration indicates a problem: Any such data must contain an infinite number of characters! In the analogy with recursive functions, this is like a recursive function without a "base case," that is, a test to stop the recursion:

```
int factorial (int n){
   return n * factorial (n – 1);       /* oops */
}
```

This function lacks the test for small `n` and results in an infinite number of calls to `factorial` (at least until memory is exhausted). We could try to remove this problem in the definition of `CharList` by providing a base case using a union:

```
union CharList{
   enum { nothing } emptyCharList; /* no data */
   struct{
      char data;
      union CharList next; /* still illegal */
   } charListNode;
};
```

Of course, this is still "wishful thinking" in that, at the line noted, the code is still illegal C. However, consider this data type as an abstract definition of what it means to be a `CharList` as a set, and then it actually makes sense, giving the following recursive equation for a `CharList`:

```
CharList = {nothing} < char 3 CharList
```

where < is union and 3 is Cartesian product. In Section 6.2.1, we described how BNF rules could be interpreted as recursive set equations that define a set by taking the smallest solution (or **least fixed point**). The current situation is entirely analogous, and one can show that the least fixed point solution of the preceding equation is:

```
{nothing} < char < (char 3 char) <
                  (char 3 char 3 char) <...
```

That is, a list is either the empty list or a list consisting of one character or a list consisting of two characters, and so on; see Exercise 8.7. Indeed, ML allows the definition of `CharList` to be written virtually the same as the above illegal C:

```
datatype CharList
   = EmptyCharList | CharListNode of char * CharList ;
```

Why is this still illegal C? The answer lies in C's data allocation strategy, which requires that each data type have a fixed maximum size determined at translation time. Unfortunately, a variable of type `CharList` has no fixed size and cannot be allocated prior to execution. This is an insurmountable obstacle to defining such a data type in a language without a fully dynamic runtime environment; see Chapter 10. The solution adopted in most imperative languages is to use pointers to allow manual dynamic allocation. Thus, in C, the direct use of recursion in type declarations is prohibited, but indirect recursive declarations through pointers is allowed, as in the following (now legal) C declarations:

```
struct CharListNode{
   char data;
   struct CharListNode* next; /* now legal */
};
typedef struct CharListNode* CharList;
```

With these declarations, each individual element in a `CharListNode` now has a fixed size, and they can be strung together to form a list of arbitrary size. Note that a union is no longer necessary, because we represent the empty list with a null pointer (the special address 0 in C). Nonempty `CharLists` are constructed using manual allocation. For example, the list containing the single character `'a'` can be constructed as follows:

```
CharList cl
   = (CharList) malloc(sizeof(struct CharListNode));
(*cl).data = 'a'; /* can also write cl->data = 'a'; */
(*cl).next = 0; /* can also write cl->next = 0; */
```

This can then be changed to a list of two characters as follows:

```
(*cl).next
     = (CharList) malloc(sizeof(struct CharListNode));
(*(*cl).next).data = 'b'; /*or cl->next->data = 'b'; */
(*(*cl).next).next = 0;  /* or cl->next->next = 0; */
```

8.3.6 Data Types and the Environment

At a number of points in this section, when the structure of a data type required space to be allocated dynamically, we have referred the reader to the discussion in Chapter 10 on environments,. This type of storage allocation is the case for pointer types, recursive types, and general function types. In their most general forms, these types require fully dynamic environments with automatic allocation and deallocation ("garbage collection") as exemplified by the functional languages and the more dynamic object-oriented languages (such as Smalltalk). More traditional languages, such as C++ and Ada, carefully restrict these types so that a heap (a dynamic address space under programmer control) suffices, in addition to a traditional stack-based approach to nested blocks (including functions), as discussed in Chapter 7. While it would make sense to discuss these general environment issues at this point and relate them directly to data structures, we opt to delay a full discussion until Chapter 10, where all environment issues, including those of procedure and function calls, can be addressed.

More generally, the requirements of recursive data and recursive functions present a duality. In a language with very general function constructs (such as the functional languages of Chapter 3), recursive data can be modeled entirely by functions. Similarly, object-oriented languages (Chapter 5) can model recursive functions entirely by recursive data objects or polymorphic methods. While we do not focus on this duality in detail in this book, specific examples can be found in Chapters 3 and 5 and their exercises.

8.4 Type Nomenclature in Sample Languages

Although we have presented the general scheme of type mechanisms in Sections 8.2 and 8.3, various language definitions use different and confusing terminology to define similar things. In this section, we give a brief description of the differences among three of the languages used in previous examples: C, Java, and Ada.

8.4.1 C

An overview of C data types is given in Figure 8.5. The simple types are called **basic types** in C, and types that are constructed using type constructors are called **derived types**. The basic types include: the `void` type (in a sense the simplest of all types, whose set of values is empty); the **numeric types**; the **integral types,** which are ordinal; and the **floating types**. There are three kinds of floating types and 12 possible kinds of integral types. Among these there are four basic kinds, all of which can also have either `signed` or `unsigned` attributes given them (indicated in Figure 8.5 by listing the four basic types with `signed` and `unsigned` in parentheses above them). Of course, not all of the 12 possible integral types are distinct. For example, `signed int` is the same as `int`. Other duplicates are possible as well.

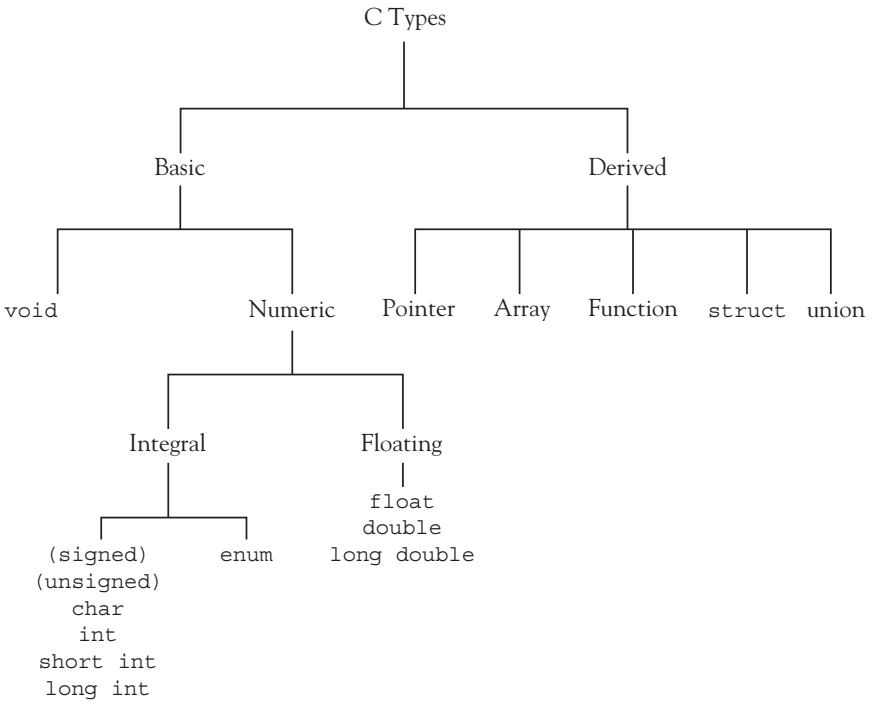

Figure 8.5 The type structure of C

8.4.2 Java

The Java type structure is shown in Figure 8.6. In Java the simple types are called **primitive types**, and types that are constructed using type constructors are called **reference types** (since they are all implicitly pointers or references). The primitive types are divided into the single type `boolean` (which

is *not* a numeric or ordinal type) and the numeric types, which split as in C into the integral (ordinal) and floating-point types (five integral and two floating point). There are only three type constructors in Java: the array (with no keyword as in C), the **class**, and the **interface**. The `class` and `interface` constructors were described in Chapter 5.

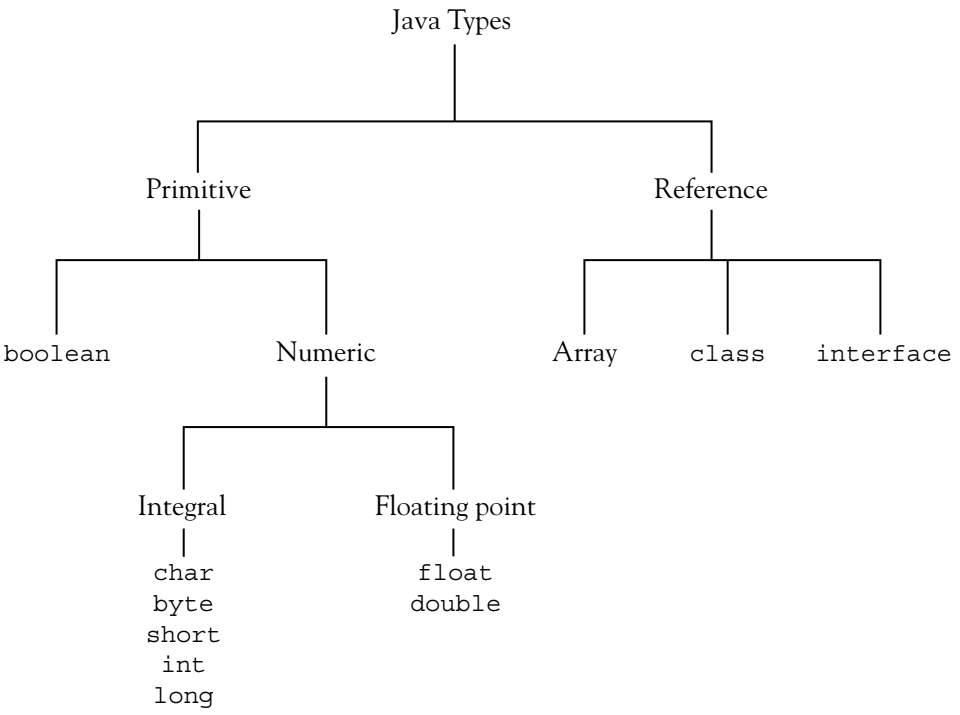

Figure 8.6 The type structure of Java

8.4.3 Ada

Ada has a rich set of types, a condensed overview of which is given in Figure 8.7. Simple types, called **scalar types** in Ada, are split into several overlapping categories. Ordinal types are called **discrete** types in Ada. **Numeric types** comprise the **real** and **integer** types. Pointer types are called `access` types. Array and record types are called **composite types**.

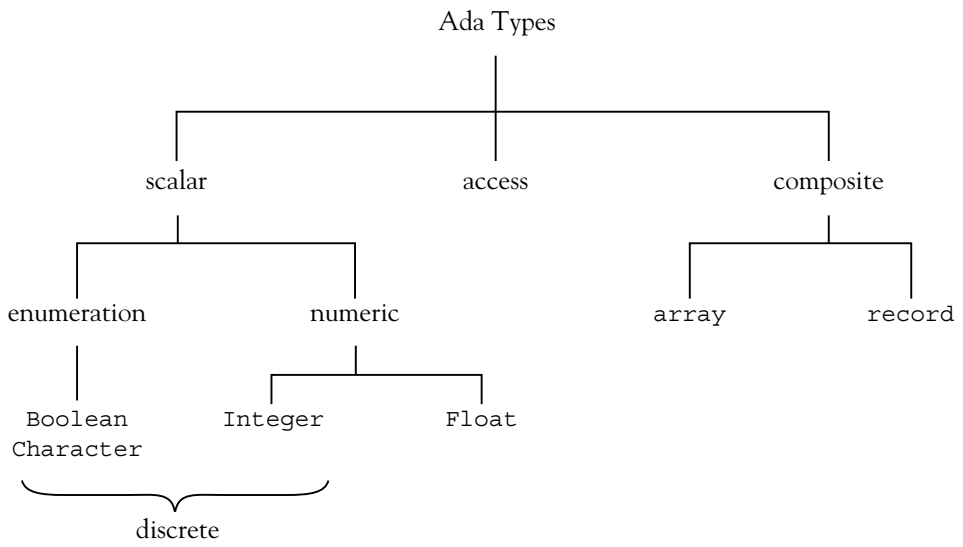

Figure 8.7 The type structure of Ada (somewhat simplified)

8.5 Type Equivalence

A major question involved in the application of types to type checking has to do with **type equivalence**: When are two types the same? One way of trying to answer this question is to compare the sets of values simply as sets. Two sets are the same if they contain the same values: For example, any type defined as the Cartesian product *A* X *B* is the same as any other type defined in the same way. On the other hand, if we assume that type *B* is not the same as type *A*, then a type defined as *A* X *B* is not the same as a type defined as *B* X *A*, since *A* X *B* contains the pairs (*a, b*) but *B* X *A* contains the pairs (*b, a*). This view of type equivalence is that two data types are the same if they have the same structure: They are built in exactly the same way using the same type constructors from the same simple types. This form of type equivalence is called **structural equivalence** and is one of the principal forms of type equivalence in programming languages.

Example

In a C-like syntax, the types `struct Rec1` and `struct Rec2` defined as follows are structurally equivalent, but `struct Rec1` and `struct Rec3` are not (the `char` and `int` fields are reversed in the definition of `struct Rec3`):

```
struct Rec1{
   char x;
   int y;
   char z[10];
};
```

(continues)

(continued)

```
struct Rec2{
   char x;
   int y;
   char z[10];
};

struct Rec3{
   int y;
   char x;
   char z[10];
};
```

Structural equivalence is relatively easy to implement (except for recursive types, as explained later in this chapter). What's more, it provides all the information needed to perform error checking and storage allocation. It is used in such languages as Algol60, Algol68, FORTRAN, COBOL, and a few modern languages such as Modula-3. It is also used selectively in such languages as C, Java, and ML, as explained later in this chapter. To check structural equivalence, a translator may represent types as trees and check equivalence recursively on subtrees (see Exercise 8.36). Questions still arise, however, in determining how much information is included in a type under the application of a type constructor. For example, are the two types `A1` and `A2` defined by

```
typedef int A1[10];
typedef int A2[20];
```

structurally equivalent? Perhaps yes, if the size of the index set is not part of an array type; otherwise, no. A similar question arises with respect to member names of structures. If structures are taken to be just Cartesian products, then, for example, the two structures

```
struct RecA{
   char x;
   int y;
};
```

and

```
struct RecB{
   char a;
   int b;
};
```

should be structurally equivalent; typically, however, they are not equivalent, because variables of the different structures would have to use different names to access the member data.

A complicating factor is the use of **type names** in declarations. As noted previously, type expressions in declarations may or may not be given explicit names. For example, in C, variable declarations may be given using **anonymous types** (type constructors applied without giving them names), but names can also be given right in `struct`s and `union`s, or by using a `typedef`. For example, consider the C code:

```
struct RecA{
   char x;
   int y;
} a;

typedef struct RecA RecA;

typedef struct{
   char x;
   int y;
} RecB;

RecB b;

struct{
   char x;
   int y;
} c;
```

Variable `a` has a data type with two names: `struct RecA` and `RecA` (as given by the `typedef`). Variable `b`'s type has only the name `RecB` (the `struct` name was left blank). And variable `c`'s type has no name at all! (Actually, `c`'s type still has a name, but it is internal and cannot be referred to by the programmer). Of course, the types `struct RecA`, `RecA`, `RecB`, and `c`'s anonymous type are all structurally equivalent.

Structural equivalence in the presence of type names remains relatively simple—simply replace each name by its associated type expression in its declaration—*except* for recursive types, where this rule would lead to an infinite loop (recall that a major reason for introducing type names is to allow for the declaration of recursive types). Consider the following example (a variation on an example used previously):

```
typedef struct CharListNode* CharList;
typedef struct CharListNode2* CharList2;

struct CharListNode{
   char data;
   CharList next;
};

struct CharListNode2{
   char data;
   CharList2 next;
};
```

Clearly `CharList` and `CharList2` are structurally equivalent, but a type checker that simply replaces type names by definitions will get into an infinite loop trying to verify it! The secret is to *assume* that `CharList` and `CharList2` are structurally equivalent to start with. It then follows easily that `CharListNode` and `CharListNode2` are structurally equivalent, and then that `CharList` and `CharList2` are indeed themselves equivalent. Of course, this seems like a circular argument, and it must be done carefully to give correct results. The details can be found in Louden [1997].

To avoid this problem, a different, much stricter, type equivalence algorithm was developed that focuses on the type names themselves. Two types are the same only if they have the same name. For obvious reasons, this is called **name equivalence**.

Example

In the following C declarations,

```
struct RecA{
   char x;
   int y;
};

typedef struct RecA RecA;

struct RecA a;
RecA b;
struct RecA c;
struct{
   char x;
   int y;
} d;
```

all of the variables `a`, `b`, `c`, and `d` are structurally equivalent. However, `a` and `c` are name equivalent (and not name equivalent to `b` or `d`), while `b` and `d` are not name equivalent to any other variable. Similarly, given the declarations

```
typedef  int Ar1[10];
typedef Ar1 Ar2;
typedef int Age;
```

types `Ar1` and `Ar2` are structurally equivalent but not name equivalent, and `Age` and `int` are structurally equivalent but not name equivalent. ■

Name equivalence in its purest form is even easier to implement than structural equivalence, as long as we *force* every type to have an explicit name (this can actually be a good idea, since it *documents* every type with an explicit name): Two types are the same only if they are the same name, and two variables are type equivalent only if their declarations use exactly the same type name. We can also invent **aliases** for types (e.g., `Age` above is an alias for `int`), and the type checker forces the programmer to keep uses of aliases distinct (this is also a very good *design tool*).

The situation becomes slightly more complex if we allow variable or function declarations to contain new types (i.e., type constructors) rather than existing type names only. Consider, for example, the following declarations:

```
struct{
   char x;
   int y;
} d,e;
```

Are `d` and `e` name equivalent? Here there are no visible type names from which to form a conclusion, so a name equivalence algorithm could say either yes or no. A language might include a rule that a combined declaration as this is equivalent to separate declarations:

```
struct{
   char x;
   int y;
} d;

struct{
   char x;
   int y;
} e;
```

In this case, new internal names are generated for each new `struct`, so `d` and `e` are clearly *not* name equivalent). On the other hand, using a single `struct` in a combined declaration could be viewed as constructing only *one* internal name, in which case `d` and `e` *are* equivalent.

Ada is one of the few languages to implement a very pure form of name equivalence. The only time Ada allows type constructors to be used in variable declarations is with the array constructor:

```
a: array (1..10) of integer;
```

The following is illegal Ada:

```
a: record
   x: integer;
   y: character;
end record;
```

Instead, we must write:

```
type IntChar is record
   x: integer;
   y: character;
end record;
a: IntChar;
```

Thus, ambiguity is avoided in Ada by requiring type names in variable and function declarations in virtually all cases. Ada views simultaneous array variable declarations without type names as having separate, inequivalent types.

Note that one small special rule is made in Ada for aliases and subtypes. According to this rule, writing:

```
type Age is integer;
```

is illegal. The Ada compiler wants to know whether we want an actual new type or just an alias that should be considered equivalent to the old type. Here we must write either:

```
type Age is new integer;
     -- Age is a completely new type
```

or:

```
subtype Age is integer;
    -- Age is just an alias for integer
```

As noted previously, the `subtype` designation is also used to create subset types, which are also not new types, but indications for runtime value checking.

C uses a form of type equivalence that falls between name and structural equivalence, and which can be loosely described as "name equivalence for `structs` and `unions`, structural equivalence for everything else." What is really meant here is that applying the `struct` or `union` type constructor creates a new, nonequivalent, type. Applying any other type constructor, or using a `typedef`, does not create a new type but one that is equivalent to every other type with the same structure (taking into account the special rule for `structs` and `unions`).

Example

```
struct A{
   char x;
   int y;
};

struct B{
   char x;
   int y;
};

typedef struct A C;
typedef C* P;
typedef struct B * Q;
typedef struct A * R;
typedef int S[10];
```

(continues)

(continued)

```
typedef int T[5];
typedef int Age;
typedef int (*F)(int);
typedef Age (*G)(Age);
```

Types `struct A` and `C` are equivalent, but they are not equivalent to `struct B`; types `P` and `R` are equivalent, but not to `Q`; types `S` and `T` are equivalent; types `int` and `Age` are equivalent, as are function types `F` and `G`.

Pascal adopts a similar rule to C, except that almost all type constructors, including arrays, pointers, and enumerations, lead to new, inequivalent types.[12] However, new names for existing type names are, as in C's `typedef`s, equivalent to the original types. (Sometimes this rule is referred to as **declaration equivalence**.) Thus, in:

```
type
   IntPtr = ^integer;
   Age = integer;
var
   x: IntPtr;
   y: ^integer;
   i: Age;
   a: integer;
```

`x` and `y` are not equivalent in Pascal, but `i` and `a` are.

Java has a particularly simple approach to type equivalence. First, there are no `typedef`s, so naming questions are minimized. Second, `class` and `interface` declarations implicitly create new type names (the same as the class/interface names), and name equivalence is used for these types. The only complication is that arrays (which cannot have type names) use structural equivalence, with special rules for establishing base type equivalence (we do not further discuss Java arrays in this text—see the Notes and References for more information).

Last, we mention the type equivalence rule used in ML. ML has *two* type declarations—`datatype` and `type`, but only the former constructs a new type, while the latter only constructs aliases of existing types (like the `typedef` of C). For example,

```
type Age = int;
datatype NewAge = NewAge int;
```

Now `Age` is equivalent to `int`, but `NewAge` is a new type not equivalent to `int`. Indeed, we reused the name `NewAge` to also stand for a data constructor for the `NewAge` type, so that `(NewAge 2)` is a value of type `NewAge`, while 2 is a value of type `int` (or `Age`).

One issue we have omitted from this discussion of type equivalence is the interaction between the type equivalence algorithm and the type-checking algorithm. As explained earlier, type equivalence

[12]Subrange types in Pascal are implemented as runtime checks, not new types.

comes up in type checking and can be given a somewhat independent answer. In fact, some type equivalence questions may never arise because of the type-checking rules, and so we might never be able to write code in a language that would answer a particular type equivalence question. In such cases, we might sensibly adopt an operational view and simply disregard the question of type equivalence for these particular cases. An example of this is discussed in Exercise 8.25.

8.6 Type Checking

Type checking, as explained at the beginning of the chapter, is the process by which a translator verifies that all constructs in a program make sense in terms of the types of its constants, variables, procedures, and other entities. It involves the application of a type equivalence algorithm to expressions and statements, with the type-checking algorithm varying the use of the type equivalence algorithm to suit the context. Thus, a strict type equivalence algorithm such as name equivalence could be relaxed by the type checker if the situation warrants.

Type checking can be divided into **dynamic** and **static** checking. If type information is maintained and checked at runtime, the checking is dynamic. Interpreters by definition perform dynamic type checking. But compilers can also generate code that maintains type attributes during runtime in a table or as type tags in an environment. A Lisp compiler, for example, would do this. Dynamic type checking is required when the types of objects can only be determined at runtime.

The alternative to dynamic typing is static typing: The types of expressions and objects are determined from the text of the program, and type checking is performed by the translator before execution. In a strongly typed language, all type errors must be caught before runtime, so these languages must be statically typed, and type errors are reported as compilation error messages that prevent execution. However, a language definition may leave unspecified whether dynamic or static typing is to be used.

Example 1

C compilers apply static type checking during translation, but C is not really strongly typed since many type inconsistencies do not cause compilation errors but are automatically removed through the generation of conversion code, either with or without a warning message. Most modern compilers, however, have error level settings that do provide stronger typing if it is desired. C++ also adds stronger type checking to C, but also mainly in the form of compiler warnings rather than errors (for compatibility with C). Thus, in C++ (and to a certain extent also in C), many type errors appear only as warnings and do not prevent execution. Thus, ignoring warnings can be a "dangerous folly" (Stroustrup [1994], p. 42). ■

Example 2

The Scheme dialect of Lisp (see Chapter 3) is a dynamically typed language, but types *are* rigorously checked, with all type errors causing program termination. There are no types in declarations, and there are no explicit type names. Variables and other symbols have no predeclared types but take on the type of the value they possess at each moment of execution. Thus, types in Scheme must be kept as explicit attributes of values. Type checking consists of generating errors for functions requiring certain values to perform their operations. For example, `car` and `cdr` require their operands to be lists: `(car 2)` generates an error. Types can be checked explicitly by the programmer, however, using predefined test functions.

Types in Scheme include lists, symbols, atoms, and numbers. Predefined test functions include `number?` and `symbol?`. (Such test functions are called **predicates** and always end in a question mark.)

Example 3

Ada is a strongly typed language, and all type errors cause compilation error messages. However, even in Ada, certain errors, like range errors in array subscripting, cannot be caught prior to execution, since the value of a subscript is not in general known until runtime. However, the Ada standard guarantees that all such errors will cause exceptions and, if these exceptions are not caught and handled by the program itself, program termination. Typically, such runtime type-checking errors result in the raising of the predefined exception `Constraint_Error`.

An essential part of type checking is **type inference**, where the types of expressions are inferred from the types of their subexpressions. Type-checking rules (that is, when constructs are type correct) and type inference rules are often intermingled. For example, an expression $e1 + e2$ might be declared type correct if $e1$ and $e2$ have the same type, and that type has a "1" operation (type checking), and the result type of the expression is the type of $e1$ and $e2$ (type inference). This is the rule in Ada, for example. In other languages this rule may be softened to include cases where the type of one subexpression is automatically convertible to the type of the other expression.

As another example of a type-checking rule, in a function call, the types of the actual parameters or arguments must match the types of the formal parameters (type checking), and the result type of the call is the result type of the function (type inference).

Type-checking and type inference rules have a close interaction with the type equivalence algorithm. For example, the C declaration

```
void p ( struct { int i; double r; } x ){
...
}
```

is an error under C's type equivalence algorithm, since no actual parameter can have the type of the formal parameter `x`, and so a type mismatch will be declared on every call to `p`. As a result, a C compiler will usually issue a warning here (although, strictly speaking, this is legal C).[13] Similar situations occur in Pascal and Ada.

The process of type inference and type checking in statically typed languages is aided by explicit declarations of the types of variables, functions, and other objects. For example, if `x` and `y` are variables, the correctness and type of the expression `x + y` is difficult to determine prior to execution unless the types of `x` and `y` have been explicitly stated in a declaration. However, explicit type declarations are not an absolute requirement for static typing: The languages ML and Haskell perform static type checking but do not require types to be declared. Instead, types are inferred from context using an inference mechanism that is more powerful than what we have described (it will be described in Section 8.8).

Type inference and correctness rules are often one of the most complex parts of the semantics of a language. Nonorthogonalities are hard to avoid in imperative languages such as C. In the remainder of this section, we will discuss major issues and problems in the rules of a type system.

[13]This has been raised to the level of an error in C++.

8.6.1 Type Compatibility

Sometimes it is useful to relax type correctness rules so that the types of components need not be precisely the same according to the type equivalence algorithm. For example, we noted earlier that the expression *e*1 + *e*2 may still make sense even if the types of *e*1 and *e*2 are different. In such a situation, two different types that still may be correct when combined in certain ways are often called **compatible** types. In Ada, any two subranges of the same base type are compatible (of course, this can result in errors, as we shall see in Section 8.6.3.). In C and Java, all numeric types are compatible (and conversions are performed such that as much information as possible is retained).

A related term, **assignment compatibility**, is often used for the type correctness of the assignment *e*1 + *e*2 (which may be different from other compatibility rules because of the special nature of assignment). Initially, this statement may be judged type correct when `x` and *e* have the same type. However, this ignores a major difference: The left-hand side must be an **l-value** or **reference**, while the right-hand side must be an **r-value**. Many languages solve this problem by requiring the left-hand side to be a variable name, whose address is taken as the l-value, and by automatically **dereferencing** variable names on the right-hand side to get their r-values. In ML this is made more explicit by saying that the assignment is type correct if the type of the left-hand side (which may be an arbitrary expression) is `ref t` (a reference to a value of type `t`), and the type of the right-hand side is `t`. ML also requires explicit dereferencing of variables used on the right-hand side:

```
val x = ref 2; (* type of x is ref int *)
x = !x + 1; (* x dereferenced using ! *)
```

As with ordinary compatibility, assignment compatibility can be expanded to include cases where both sides do not have the same type. For example, in Java the assignment `x` = *e* is legal when *e* is a numeric type whose value can be converted to the type of `x` without loss of information (for example, `char` to `int`, or `int` to `long`, or `long` to `double`, etc.). On the other hand, if *e* is a floating-point type and *x* is an integral type, then this is a type error in Java (thus, assignment compatibility is different from the compatibility of other arithmetic operators). Such assignment compatibilities may or may not involve the conversion of the underlying values to a different format; see Section 8.7.

8.6.2 Implicit Types

As noted at the beginning of this chapter, types of basic entities such as constants and variables may not be given explicitly in a declaration. In this case, the type must be inferred by the translator, either from context information or from standard rules. We say that such types are **implicit**, since they are not explicitly stated in a program, though the rules that specify their types must be explicit in the language definition.

As an example of such a situation in C, variables are implicitly integers if no type is given,[14] and functions implicitly return an integer value if no return type is given:

```
x; /* implicitly integer */

f(int x) /* implicitly returns an int */
{ return x + 1; }
```

[14]This has been made illegal by the 1999 ISO C Standard.

As another example, in Pascal named constants are implicitly typed by the literals they represent:

```
const
   PI = 3.14156; (* implicitly real *)
   Answer = 42; (* implicitly integer *)
```

Indeed, literals are the major example of implicitly typed entities in all languages. For example, `123456789` is implicitly an `int` in C, unless it would overflow the space allocated for an `int`, in which case it is a `long int` (or perhaps even an `unsigned long int`, if necessary). On the other hand, `1234567L` is a `long int` by virtue of the "L" suffix. Similarly, any sequence of characters surrounded by double quotation marks such as `"Hello"` (a "C string") is implicitly a character array of whatever size is necessary to hold the characters plus the delimiting null character that ends every such literal. Thus, `"Hello"` is of type `char[6]`.

When subranges are available as independent types in a language (such as Ada), new problems arise in that literals must now be viewed as potentially of several different types. For instance, if we define:

```
type Digit_Type is range 0..9;
```

then the number 0 could be an `integer` or a `Digit_Type`. Usually in such cases the context of use is enough to tell a translator what type to choose. Alternatively, one can regard 0 as immediately convertible from `integer` to `Digit_Type`.

8.6.3 Overlapping Types and Multiply-Typed Values

As just noted, types may overlap in that two types may contain values in common. The `Digit_Type` example above shows how subtyping and subranges can create types whose sets of values overlap. Normally, it is preferable for types to be disjoint as sets, so that every expressible value belongs to a unique type, and no ambiguities or context sensitivities arise in determining the type of a value. However, enforcing this rule absolutely would be far too restrictive and would, in fact, eliminate one of the major features of object-oriented programming—the ability to create subtypes through inheritance that refine existing types yet retain their membership in the more general type. Even at the most basic level of predefined numeric types, however, overlapping values are difficult to avoid. For instance, in C the two types `unsigned int` and `int` have substantial overlap, and in Java, a small integer could be a `short`, an `int`, or a `long`. Thus, rules such as the following are necessary (Arnold, Gosling, and Holmes [2000], p. 143):

> Integer constants are long if they end in `L` or `l`, such as `29L`; `L` is preferred over `l` because `l` (lowercase `L`) can easily be confused with `1` (the digit one). Otherwise, integer constants are assumed to be of type `int`. If an int literal is directly assigned to a `short`, and its value is within the valid range for a `short`, the integer literal is treated as if it were a `short` literal. A similar allowance is made for integer literals assigned to `byte` variables.

Also, as already noted, subranges and subtypes are the source of range errors during execution that cannot be predicted at translation time and, thus, a source of program failure that type systems were in part designed to prevent. For example, the Ada declarations

```
subtype SubDigit is integer range 0..9;
subtype NegDigit is integer range −9..−1;
```

create two subranges of integers that are in fact disjoint, but the language rules allow code such as:

```
x: SubDigit;
y: NegDigit;
...
x := y;
```

without a compile-time error. Of course, this code cannot execute without generating a `CONSTRAINT_ERROR` and halting the program. Similar results hold for other languages that allow subranges, like Pascal and Modula-2.

Sometimes, however, overlapping types and multiply-typed values can be of very direct benefit in simplifying code. For example, in C the literal `0` is not only a value of every integral type but is also a value of every pointer type, and represents the null pointer that points to no object. In Java, too, there is a single literal value `null` that is, in essence, a value of every reference type.[15]

8.6.4 Shared Operations

Each type is associated, usually implicitly, with a set of operations. Often these operations are shared among several types or have the same name as other operations that may be different. For example, the operator + can be real addition, integer addition, or set union. Such operators are said to be **overloaded**, since the same name is used for different operations. In the case of an overloaded operator, a translator must decide which operation is meant based on the types of its operands. In Chapter 7 we studied in some detail how this can be done using the symbol table. In that discussion, we assumed the argument types of an overloaded operator are disjoint as sets, but problems can arise if they are not, or if subtypes are present with different meanings for these operations. Of course, there should be no ambiguity for built-in operations, since the language rules should make clear which operation is applied. For example, in Java, if two operands are of integral type, then integer addition is *always* applied (and the result is an `int`) regardless of the type of the operands, *except* when one of the operands has type `long`, in which case `long` addition is performed (and the result is of type `long`). Thus, in the following Java code, 40000 is printed (since the result is an `int`, not a `short`):

```
short x = 20000;
System.out.println(x + x);
```

[15]This is not quite accurate, according to the Java Language specification (Gosling et al. [2000], p. 32): "There is also a special null type, the type of the expression `null`, which has no name. Because the null type has no name, it is impossible to declare a variable of the null type or to cast to the null type. The null reference is the only possible value of an expression of null type. The null reference can always be cast to any reference type. In practice, the programmer can ignore the null type and just pretend that `null` is merely a special literal that can be of any reference type."

On the other hand,

```
x = x + x;
```

is now illegal in Java—it requires a cast (see next section):

```
x = (short) (x + x);
```

Thus, there is no such thing as `short` addition in Java, nor does Java allow other kinds of arithmetic on any integral types other than `int` and `long`.

Ada, as usual, allows much finer control over what operations are applied to data types, as long as these are indeed new types and not simply subranges. (Subranges simply inherit the operations of their base types.) Consider, for example, the following Ada code:

```
type Digit_Type is range 0..9;

function "+"(x,y: Digit_Type) return Digit_Type is
   a: integer := integer(x);
   b: integer := integer(y);
begin
   return Digit_Type((a+b) mod 10);
end "+";
```

This defines a new addition operation on `Digit_Type` that wraps its result so that it is always a digit. Now, given the code

```
a: Digit_Type := 5+7;
```

it may seem surprising, but Ada knows enough here to call the new `"+"` function for `Digit_Type`, rather than use standard integer addition, and a gets the value 2 (otherwise a `Constraint_Error` would occur here). The reason is (as noted in Chapter 7) that since there are no implicit conversions in Ada, Ada can use the result type of a function call, as well as the parameter argument types, to resolve overloading, and since the result type is `Digit_Type`, the user-defined `"+"` must be the function indicated.

C++ does not allow this kind of overloading. Instead, operator overloading must involve new, user-defined types. For example, the following definition:

```
int * operator+(int* a, int* b){ // illegal in C++
   int * x = new int;
   *x = *a+*b;
   return x;
}
```

is illegal in C++, as would be any attempt to redefine the arithmetic operators on any built-in type.

8.7 Type Conversion

In every programming language, it is necessary to convert one type to another under certain circumstances. Such **type conversion** can be built into the type system so that conversions are performed automatically, as in a number of examples discussed previously. For example, in the following C code:

```
int x = 3;
...
x = 2.3 + x / 2;
```

at the end of this code x still has the value 3: `x / 2` is integer division, with result 1, then 1 is converted to the `double` value 1.0 and added to 2.3 to obtain 3.3, and then 3.3 is truncated to 3 when it is assigned to x.

In this example, two automatic, or **implicit conversions** were inserted by the translator. The first is the conversion of the `int` result of the division to `double` (1 → 1.0) before the addition .The second is the conversion of the `double` result of the addition to an `int` (3.3 → 3) before the assignment to x. Such implicit conversions are sometimes also called **coercions**. The conversion from `int` to `double` is an example of a **widening** conversion, where the target data type can hold all of the information being converted without loss of data, while the conversion from `double` to `int` is a **narrowing** conversion that may involve a loss of data.

Implicit conversion has the benefit of making it unnecessary for the programmer to write extra code to perform an obvious conversion. However, implicit conversion has drawbacks as well. For one thing, it can weaken type checking so that errors may not be caught. This compromises the strong typing and the reliability of the programming language. Implicit conversions can also cause unexpected behavior in that the programmer may expect a conversion to be done one way, while the translator actually performs a different conversion. A famous example in (early) PL/I is the expression

```
1/3 + 15
```

which was converted to the value 5.33333333333333 (on a machine with 15-digit precision): The leading 1 was lost by overflow because the language rules state that the precision of the fractional value must be maintained.

An alternative to implicit conversion is **explicit conversion**, in which conversion directives are written right into the code. Such conversion code is typically called a **cast** and is commonly written in one of two varieties of syntax. The first variety is used in C and Java and consists of writing the desired result type inside parentheses before the expression to be converted. Thus, in C we can write the previous implicit conversion example as explicit casts in the following form:

```
x = (int) (2.3 + (double) (x / 2));
```

The other variety of cast syntax is to use function call syntax, with the result type used as the function name and the value to be converted as the parameter argument. C++ and Ada use this form. Thus, the preceding conversions would be written in C++ as:

```
x = int( 2.3 + double( x / 2 ) );
```

C++ also uses a newer form for casts, which will be discussed shortly.

The advantange to using casts is that the conversions being performed are documented precisely in the code, with less likelihood of unexpected behavior, or of human readers misunderstanding the code. For this reason, a few languages prohibit implicit conversions altogether, forcing programmers to write out casts in all cases. Ada is such a language. Additionally, eliminating implicit conversions makes it easier for the translator (and the reader) to resolve overloading. For example, in C++ if we have two function declarations:

```
double max (int, double);
double max (double,int);
```

then the call `max(2,3)` is ambiguous because of the possible implicit conversions from `int` to `double` (which could be done either on the first or second parameter). If implicit conversions were not allowed, then the programmer would be forced to specify which conversion is meant. (A second example of this phenomenon was just seen in Ada in the previous section.)

A middle ground between the two extremes of eliminating all implicit conversions and allowing conversions that could potentially be errors is to allow only those implicit conversions that are guaranteed not to corrupt data. For example, Java follows this principle and only permits widening implicit conversions for arithmetic types. C++, too, adopts a more cautious approach than C toward narrowing conversions, generating warning messages in all cases except the `int` to `char` narrowing implicit conversion.[16]

Even explicit casts need to be restricted in various ways by languages. Often, casts are restricted to simple types, or even to just the arithmetic types. If casts are permitted for structured types, then clearly a restriction must be that the types have identical sizes in memory. The translation then merely **reinterprets** the memory as a different type, without changing the bit representation of the data. This is particularly true for pointers, which are generally simply reinterpreted as pointing to a value of a different type. This kind of explicit conversion also allows for certain functions, such as memory allocators and deallocators, to be defined using a "generic pointer." For example, in C the `malloc` and `free` functions are declared using the generic or **anonymous pointer type** `void*`:

```
void* malloc (size_t );
void free( void*);
```

and casts may be used to convert from any pointer type[17] to `void*` and back again:

```
int* x = (int*) malloc(sizeof(int));
...
free( (void*)x );
```

Actually, C allows both of these conversions to be implicit, while C++ allows implicit conversions from pointers to `void*`, but not vice versa.

Object-oriented languages also have certain special conversion requirements, since inheritance can be interpreted as a subtyping mechanism, and conversions from subtypes to supertypes and back are necessary in some cases. This kind of conversion was discussed in more detail in Chapter 5. However, we mention here that C++, because of its mixture of object-oriented and nonobject-oriented features, as well as its compatibility with C, identifies four different kinds of casts, and has invented new syntax for each of these casts: `static_cast`, `dynamic_cast`, `reinterpret_cast`, and `const_cast`. The first, `static_cast`, corresponds to the casts we have been describing here[18] (static, since the compiler interprets the cast, generates code if necessary, and the runtime system is not involved). Thus, the previous explicit cast example would be written using the C++ standard as follows:

```
x = static_cast<int>(2.3 + static_cast<double>(x / 2));
```

An alternative to casts is to allow predefined or library functions to perform conversions. For example, in Ada casts between integers and characters are not permitted. Instead, two predefined

[16]Stroustrup [1994], pp. 41–43, discusses why warnings are not generated in this case.
[17]Not "pointers to functions," however.
[18]The `reinterpret_cast` is not used for `void*` conversions, but for more dangerous conversions such as `char*` to `int*` or `int` to `char*`.

functions (called **attribute functions** of the `character` data type) allow conversions between characters and their associated ASCII values:

```
character'pos('a') -- returns 97, the ASCII value of 'a'
character'val(97) -- returns the character 'a'
```

Conversions between strings and numbers are also often handled by special conversion functions. For example, in Java the `Integer` class in the `java.lang` library contains the conversion functions `toString`, which converts an `int` to a `String`, and `parseInt`, which converts a `String` to an int:

```
String s = Integer.toString(12345); // s becomes "12345"
int i = Integer.parseInt("54321"); // i becomes 54321
```

A final mechanism for converting values from one type to another is provided by a loophole in the strong typing of some languages. Undiscriminated unions can hold values of different types, and without a discriminant or tag (or without runtime checking of such tags) a translator cannot distinguish values of one type from another, thus permitting indiscriminate reinterpretation of values. For example, the C++ declaration

```
union{
   char c;
   bool b;
} x;
```

and the statements

```
x.b = true;
cout << static_cast<int>(x.c) << endl;
```

would allow us to see the internal value used to represent the Boolean value `true` (in fact, the C++ standard says this should always be 1).

8.8 Polymorphic Type Checking

Most statically typed languages require that explicit type information be given for all names in each declaration, with certain exceptions about implicitly typed literals, constants, and variables noted previously. However, it is also possible to use an extension of type inference to determine the types of names in a declaration *without explicitly giving those types*. Indeed, a translator can either: (1) collect information on *uses* of a name, and infer from the set of all uses a probable type that is the most general type for which all uses are correct, or (2) declare a type error because some of the uses are incompatible with others. In this section, we will give an overview of this kind of type inference and type checking, which is called **Hindley-Milner type checking** for its coinventors (Hindley [1969], Milner [1978]). Hindley-Milner type checking is a major feature of the strongly typed functional languages ML and Haskell.

First, consider how a conventional type checker works. For purposes of discussion, consider the expression (in C syntax) `a[i] + i`. For a normal type checker to work, both `a` and `i` must be declared, `a` as an array of integers, and `i` as an integer, and then the result of the expression is an integer.

Syntax trees show this in more detail. The type checker starts out with a tree such as the one shown in Figure 8.8.

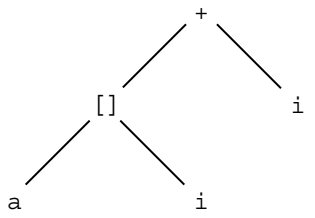

Figure 8.8 Syntax tree for a[i] + i

First, the types of the names (the leaf nodes) are filled in from the declarations, as shown in Figure 8.9.

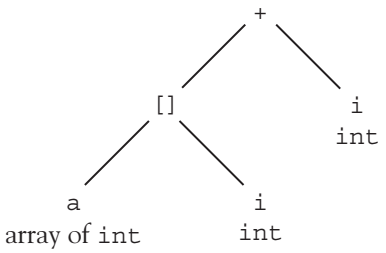

Figure 8.9 Types of the names filled in from the declarations

The type checker then checks the subscript node (labeled `[]`); the left operand must be an array, and the right operand must be an `int`; indeed, they are, so the operation type checks correctly. Then the inferred type of the subscript node is the component type of the array, which is `int`, so this type is added to the tree, as shown in Figure 8.10.

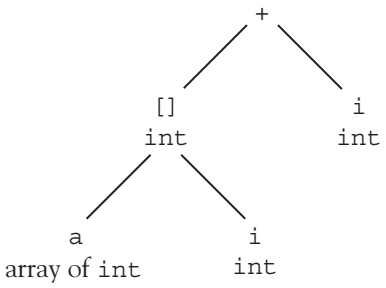

Figure 8.10 Inferring the type of the subscript node

Finally, the + node is type checked; the two operands must have the same type (we assume no implicit conversions here), and this type must have a + operation. Indeed, they are both `int`, so the + operation is type correct. Thus, the result is the type of the operands, which is `int`, as shown in Figure 8.11.

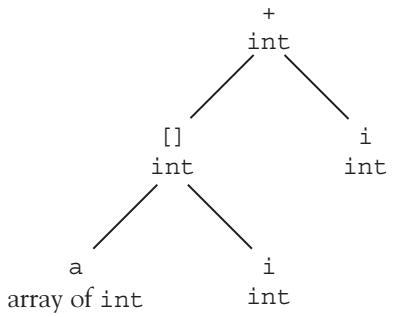

Figure 8.11 Inferring the type of the + node

Could the type checker have come to any other conclusion about the types of the five nodes in this tree? No, not even if the declarations of `a` and `i` were missing. Here's how it would do it.

First, the type checker would assign **type variables** to all the names for which it did not already have types. Thus, if `a` and `i` do not yet have types, we need to assign some type variable names to them. Since these are internal names that should not be confused with program identifiers, let us use a typical convention and assign the Greek letters α and β as the type variables for `a` and `i`. See Figure 8.12.

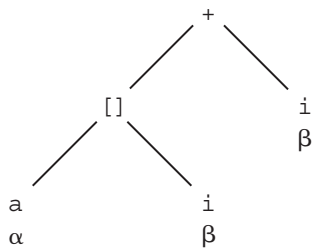

Figure 8.12 Assigning type variable to names without types

Note in particular that both occurrences of `i` have the same type variable β.

Now the type checker visits the subscript node, and infers that, for this to be correct, the type of `a` must actually be an array (in fact, array of γ, where γ[19] is another type variable, since we don't yet have any information about the component type of `a`). Also, the type checker infers that the type of `i` must be `int` (we assume that the language only allows `int` subscripts in this example). Thus, β is replaced by `int` in the entire tree, and the situation looks like Figure 8.13.

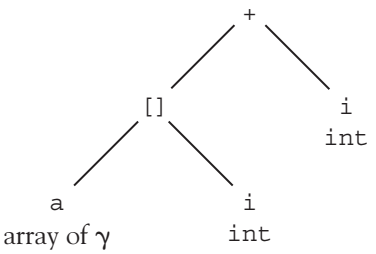

Figure 8.13 Inferring the type of the variable a

[19]Greek letter gamma.

Finally, the type checker concludes that, with these assignments to the type variables, the subscript node is type correct and has the type γ (the component type of a). See Figure 8.14.

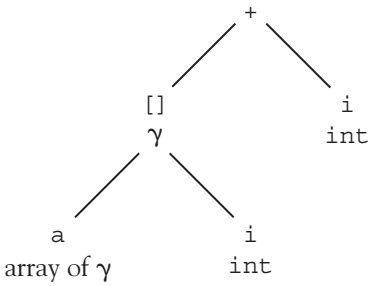

Figure 8.14 Inferring the type of the subscript node

Now the type checker visits the + node, and concludes that, for it to be type correct, γ must be `int` (the two operands of + must have the same type). Thus γ is replaced by `int` *everywhere* it appears, and the result of the + operation is also `int`. Note that we have arrived at the same typing of this expression as before, without using any prior knowledge about the types of a and i. See Figure 8.15.

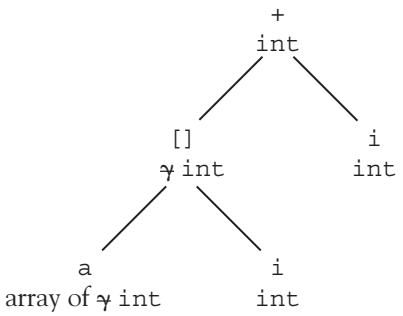

Figure 8.15 Substituting int for the type variable

This is the basic form of operation of Hindley-Milner type checking.

We note two things about this form of type checking. First, as we have indicated, once a type variable is replaced by an actual type (or a more specialized form of a type), then *all* instances of that variable must be updated to that same new value for the type variable. This process is called **instantiation** of type variables, and it can be achieved using various forms of indirection into a table of type expressions that is very similar to a symbol table. The second thing to note is that when new type information becomes available, the type expressions for variables can change form in various ways. For example, in the previous example α became "array of γ," β became `int`, and then γ became `int` as well. This process can become even more complex. For example, we might have two type expressions "array of α" and "array of β" that need to be the same for type checking to succeed. In that case, we must have α == β, and so β must be changed to α everywhere it occurs (or vice versa). This process is called **unification**; it is a kind of pattern matching that appears frequently in programming languages: Versions of it exist in several contexts in functional languages like ML and Haskell (to handle type-checking and

pattern-matching code), logic languages like Prolog (to handle variables and to direct execution), and even C++ compilers (to handle template instantiation). These other uses will be discussed elsewhere in this book. Unification involves essentially three cases (here described in terms of types):

1. Any type variable unifies with any type expression (and is instantiated to that expression).
2. Any two type constants (that is, specific types like `int` or `double`) unify only if they are the same type.
3. Any two type constructions (that is, applications of type constructors like "array" or `struct`) unify only if they are applications of the same type constructor and all of their component types also (recursively) unify.

For example, unifying type variable β with type expression "array of α" is case 1 above, and β is instantiated to "array of α." Unifying `int` to `int` is an example of case 2. Unifying "array of α" with "array of β" is case 3, and we must recurse on the component types of each expressions (which are α and β, and thus unify by case 1). On the other hand, by case 2, `int` cannot unify with "array of α," and by case 3, "array of `int`" cannot unify with "array of `char`."

Hindley-Milner type checking can be extremely useful in that it simplifies tremendously the amount of type information that the programmer must write down in code.[20] But if this were the only advantage it would not be an overwhelming one (see previous footnote). In fact, Hindley-Milner type checking gives an enormous advantage over simple type checking in that *types can remain as general as possible*, while still being strongly checked for consistency.

Consider, for example, the following expression (again in C syntax):

```
a[i] = b[i]
```

This assigns the value of `b[i]` to `a[i]` (and returns that value). Suppose again that we know nothing in advance about the types of `a`, `b`, and `i`. Hindley-Milner type checking will then establish that `i` must be an `int`, `a` must be an "array of α," and `b` must be an "array of β," and then (because of the assignment, and assuming no implicit conversions), α == β. Thus, type checking concludes with the types of `a` and `b` narrowed to "array of α," but α is still a completely unconstrained type variable that could be *any* type. This expression is said to be **polymorphic**, and Hindley-Milner type checking implicitly implements **polymorphic type checking**—that is, we get such polymorphic type checking "for free" as an automatic consequence of the implicit type variables introduced by the algorithm.

In fact, there are several different interpretations for the word "polymorphic" that we need to separate here. Polymorphic is a Greek word meaning "of many forms." In programming languages, it applies to names that (simultaneously) can have more than one type. We have already seen situations in which names can have several different types simultaneously—namely, the **overloaded functions**. So overloading is one kind of polymorphism. However, overloaded functions have only a finite (usually small) set of types, each one given by a different declaration. The kind of polymorphism we are seeing here is different: The type "array of α" is actually a set of *infinitely many* types, depending on the (infinitely many) possible instantiations of the type variable α. This

[20]Of course, this must be balanced with the advantage of documenting the desired types directly in the code: It is often useful to write explicit type information even when unnecessary in a language like ML, for documentation purposes.

type of polymorphism is called **parametric polymophism** because α is essentially a *type parameter* that can be replaced by any type expression. In fact, so far in this section, you have seen only **implicit parametric polymorphism**, since the type parameters (represented by Hindley-Milner type variables) are *implicitly introduced* by the type checker, rather than being explicitly written by the programmer. (In the next section, we will discuss **explicit parametric polymorphism**.) To distinguish overloading more clearly from parametric polymorphism, it is sometimes called **ad hoc polymorphism**. *Ad hoc* means "to this" in Latin and refers to anything that is done with a specialized goal or purpose in mind. Indeed, overloading is always represented by a set of distinct declarations, each with a special purpose. Finally, a third form of polymorphism occurs in object-oriented languages: Objects that share a common ancestor can also either share or redefine the operations that exist for that ancestor. Some authors refer to this as **pure polymorphism**. However, in keeping with modern terminology, we shall call it **subtype polymorphism**, since it is closely related to the view of subtypes as sharing operations (see Section 8.3.3). Subtype polymorphism was discussed in Chapter 5.

Finally, a language that exhibits no polymorphism, or an expression or function that has a unique, fixed type, is said to be **monomorphic** (Greek for "having one form").

The previous example of a polymorphic expression was somewhat too simple to show the true power of parametric polymorphism and Hindley-Milner type checking. Polymorphic functions are the real goal of this kind of polymorphism, so consider an example that appeared in the discussion of overloading in Chapter 7—namely, that of a `max` function such as:

```
int max (int x, int y){
   return x > y ? x : y;
}
```

We note two things about this function. First, the body for this function is the same, regardless of the type (`double` or any other arithmetic type could be substituted for `int` without changing the body). Second, the body *does* depend on the existence of a greater-than operator > for each type used in overloading this function, and the > function itself is overloaded for a number of types. Thus, to *really* remove the dependency of this function on the type of its parameters, we should add a new parameter representing the greater-than function:[21]

```
int max (int x, int y, int (*gt)(int a,int b) ) {
   return gt(x,y) ? x : y;
}
```

Now let us consider a version of this function that does not specify the type of `x`, `y`, or `gt`. We could write this in C-like syntax as:

```
max (x, y, gt) {
   return gt(x,y) ? x : y;
}
```

[21] Actually, leaving the dependency on the > function is also possible; it is called **constrained parametric polymorphism** and will be briefly discussed in the next section.

or, to use the syntax of ML (where this is legal code):

```
fun max (x, y, gt) = if gt(x,y) then x else y;
```

Let us invent a syntax tree for this definition and assign type variables to the identifiers, as shown in Figure 8.16.

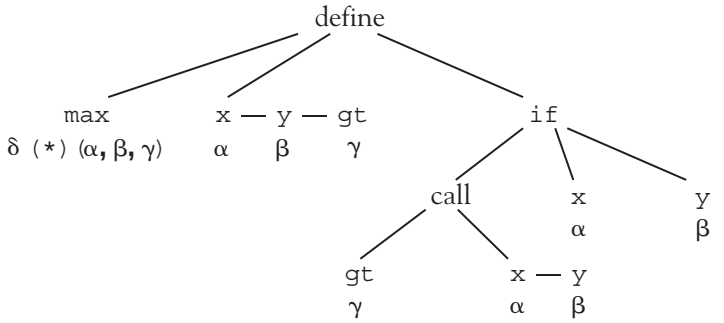

Figure 8.16 A syntax tree for the function max

Note that we are using C notation for the type of the function `max`, which is a function of three parameters with types α, β, γ, and returns a value of type δ.

We now begin type checking, and visit the call node. This tells us that `gt` is actually a function of two parameters of types α and β, and returns a type ε (which is as yet unknown). Thus, γ = ε (*)(α, β) and we get the tree shown in Figure 8.17.

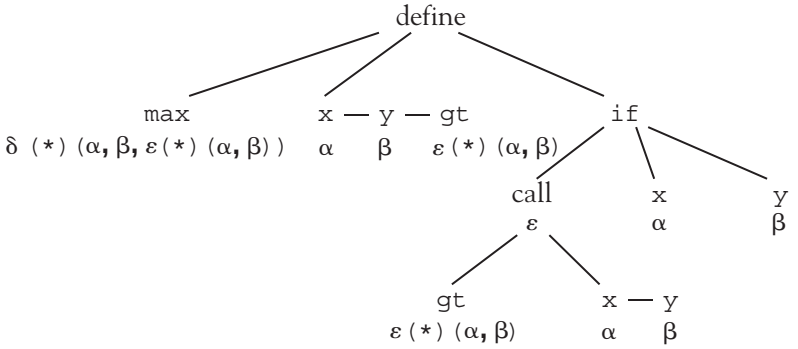

Figure 8.17 Substituting for a type variable

Note that the result of the call has type ε, the return type of the `gt` function.

Now type checking continues with the `if` node. Typically, such a node requires that the test expression return a Boolean value (which doesn't exist in C), and that the types of the "then" part and the "else" part must be the same. (C will actually perform implicit conversions in a `?:` expression, if necessary.) We will assume the stronger requirements (and the existence of a `bool` type), and we get that α = β = δ and ε = `bool`, with the tree shown in Figure 8.18.

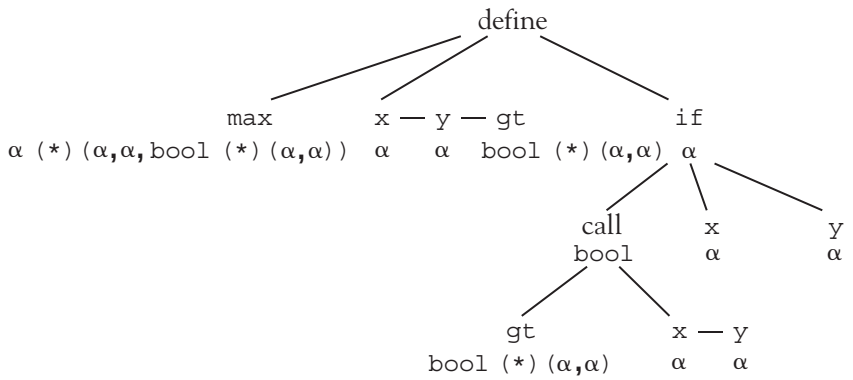

Figure 8.18 Further substitutions

($\delta = \alpha$ because the result type of the if expression is now α, and this is the type of the body of the function; hence the return type of the body.) Thus, `max` has been determined to have type α `(*)(`α, α, `bool (*)(`α, α`))`, where α is *any* type—in other words, a type parameter that can have any actual type substituted for it. In ML this type is printed as:

```
'a * 'a * ('a * 'a -> bool) -> 'a
```

where `'a` is used instead of the type variable α, a function type is indicated by the arrow `->`, and the multiple parameters are represented by a tuple (or Cartesian product) type[22] (the Cartesian product `A` $\times$ `B` in ML is printed as `A*B` and values of this type are written as tuples `(a, b)` with $\alpha \in$ `A` and `b` $\in$ `B`).

With this typing for the function `max`, we can use `max` in any situation where the actual types unify with the polymorphic type. For example, if we provide the following definitions in ML:

```
fun gti (x:int,y) = x > y;
fun gtr (x:real,y) = x > y;
fun gtp ((x,y),(z,w)) = gti (x,z);
```

we can call the `max` function as follows:

```
max(3,2,gti); (* returns 3 *)
max(2.1,3.2,gtr); (* returns 3.2 *)
max((2,"hello"),(1,"hi"),gtp); (* returns (2,"hello") *)
```

Note that α `(*)(`α, α, `bool (*)(`α, α`))` really is the most general type possible for the `max` function with the given implementation (called its **principal type**), and that each call to `max` **specializes** this principle type to a monomorphic type, which may also implicitly specialize the types of the parameters. For example, the type of the `gtp` function is `bool (*) ((int,` α`),(int,` β`))` (or `(int * 'a) * (int * 'b) -> bool` in ML syntax), with two type variables for the types of `y` and `w`, since these two variables are not used in the body of `gtp`, so each can be of any type. Now, in the call to `max` above, we still could not use two different types for `y` and `w`, since the `max` function insists on the same type for the first two parameters:

[22]In fact, we have written the parameters (`x, y, gt`) in tuple notation in the definition. It is possible to write them differently, resulting in what is called a Curried function. This was discussed in Chapter 4.

```
max((2,"hello"),(1,2),gtp); (* illegal! *)
```

so in any call to `max` the `gtp` function is specialized to the type `bool(*) ((int,α),(int,α))`.

An extension of this principle is that any polymorphically typed object that is passed into a function as a parameter must have a fixed specialization for the duration of the function. Thus, if `gtp` were called again within the `max` function, it would still have to retain the above specialization. The same is not true for nonlocal references to polymorphic functions. For example, the following code is legal ML:

```
fun ident x = x;
fun makepair (x,y) = (ident x,ident y);
makepair(2,3.2);
 (* ok -- ident has two different types, one for each call *)
```

However, making `ident` into a parameter results in a type error:

```
fun makepair2 (x,y,f) = (f x, f y);
makepair2(2,3.2,ident);
      (* type error -- ident cannot have general type *)
```

This restriction on polymorphic types in Hindley-Milner type systems is called **let-bound polymorphism**.

Let-bound polymorphism complicates Hindley-Milner type checking because it requires that type variables be divided into two separate categories. The first category is the one in use during type checking, whose value is everywhere the same and is universally changed with each instantiation/specialization. The second is part of a polymorphic type in a nonlocal reference, which can be instantiated differently at each use. Sometimes these latter type variables are called **generic** type variables. Citations for further detail on this issue can be found in the Notes and References.

An additional problem with Hindley-Milner type checking arises when an attempt is made to unify a type variable with a type expression that contains the variable itself. Consider the following function definition (this time in ML syntax):

```
fun f (g) = g (f);
```

Let's examine what happens to this definition during type checking. First, `g` will be assigned the type variable α, and `f` will be assigned the type $\alpha \rightarrow \beta$ (a function of one parameter of type α returning a value of type β, using ML-like syntax for function types). Then the body of `f` will be examined, and it indicates that `g` is a function of one parameter whose type must be the type of `f`, and whose return type is also the return type of `f`. Thus, α (the type of `g`) should be set equal to $(\alpha \rightarrow \beta) \rightarrow \beta$. However, trying to establish the equation $\alpha = (\alpha \rightarrow \beta) \rightarrow \beta$ will cause an infinite recursion, because α will contain itself as a component. Thus, before any unification of a type variable with a type expression is attempted, the type checker must make sure that the type variable is not itself a part of the type expression it is to become equal to. This is called the **occur-check**, and it must be a part of the unification algorithm for type checking to operate correctly (and declare such a situation a type error). Unfortunately, this is a difficult check to perform in an efficient way, and so in some language contexts where unification is used (such as in versions of Prolog) the occur-check is omitted. However, for Hindley-Milner type checking to work properly, the occur-check is essential.

Finally, we want to mention the issue of translating code with polymorphic types. Consider the previous example of a polymorphic `max` function. Clearly the values `x` and `y` of arbitrary type will need to be copied from location to location by the code, since `gt` is to be called with arguments `x` and `y`, and then either `x` or `y` is to be returned as the result. However, without knowing the type of `x` (and `y`), a translator cannot determine the size of these values, which must be known in order to make copies. How can a translator effectively deal with the problem?

There are several solutions. Two standard ones are as follows:

1. *Expansion*. Examine all the calls of the `max` function, and generate a copy of the code for each different type used in these calls, using the appropriate size for each type.
2. *Boxing and tagging*. Fix a size for all data that can contain any scalar value such as integers, floating-point numbers, and pointers; add a bit field that tags a type as either scalar or not, with a size field if not. All structured types then are represented by a pointer and a size, and copied indirectly, while all scalar types are copied directly.

In essence, solution 1 makes the code look as if separate, overloaded functions had been defined by the programmer for each type (but with the advantage that the compiler generates the different versions, not the programmer). This is efficient but can cause the size of the target code to grow very large (sometimes called **code bloat**), unless the translator can use target machine properties to combine the code for different types. Solution 2 avoids code bloat but at the cost of many indirections, which can substantially affect performance. Of course, this indirection may be necessary anyway for purposes of memory management, but there is also the increase in data size required by the tag and size fields, even for simple data.

8.9 Explicit Polymorphism

Implicit parametric polymorphism is fine for defining polymorphic functions, but it doesn't help us if we want to define polymorphic data structures. Suppose, for example, that we want to define a stack that can contain any kind of data (here implemented as a linked list):

```
typedef struct StackNode{
   ?? data;
   struct StackNode * next;
} * Stack;
```

Here the double question mark stands for the type of the data, which must be written in for this to be a legal declaration. Thus, to define such a data type, it is impossible to make the polymorphic type *implicit*; instead, we must write the type variable *explicitly* using appropriate syntax. This is called **explicit parametric polymorphism**.

Explicit parametric polymorphism is allowed in ML, of course. It would be foolish to have implicit parametric polymorphism for functions without explicit parametric polymorphism for data structures.

The same notation is used for type parameters as in the types for implicitly polymorphic functions: `'a`, `'b`, `'c` can be used in data structure declarations to indicate arbitrary types. Here is a `Stack` declaration in ML that imitates the above C declaration, but with an explicit type parameter:

```
datatype 'a Stack = EmptyStack
                  | Stack of 'a * ('a Stack);
```

First, note that this is unusual syntax, in that the type parameter `'a` is written *before* the type name (`Stack`) rather than after it (presumably in imitation of what one does without the parameter, which is to define various stacks such as `IntStack`, `CharStack`, `StringStack`, etc.). Second, this declaration incorporates the effect of the pointer in C by expressing a stack as a union of the empty stack with a stack containing a pair of components—the first the data (of type `'a`), and the second another stack (the tail or rest of the original stack). We can now write values of type `Stack` as follows:

```
val empty = EmptyStack; (* empty has type 'a Stack *)
val x = Stack(3, EmptyStack); (*x has type int Stack *)
```

Note that `EmptyStack` has no values, so is still of polymorphic type, while `x`, which contains the single value 3, is specialized by this value to `int Stack`.

Explicit parametric polymorphism creates no special new problems for Hindley-Milner type checking. It can operate in the same way on explicitly parameterized data as it did on implicitly parameterized functions. Indeed, polymorphic functions can be declared using an explicitly parameterized type, as for instance:

```
fun top (Stack p) = #1(p);
```

which (partially) defines `top` as a function `'a Stack -> 'a` by using the projection function `#1` on the pair `p` to extract its first component, which is the data. Thus, for the `x` previously defined, `top x` = 3.

Explicitly parameterized polymorphic data types fit nicely into languages with Hindley-Milner polymorphic type checking, but in fact they are nothing more than a mechanism for creating **user-defined type constructors**.

Recall that a type constructor, like array or `struct`, is a *function* from types to types, in that types are supplied as parameter values (these are the component types), and a new type is returned. For example, the array constructor in C, denoted by the brackets `[ ]`, takes a component type (`int`, say) as a parameter and constructs a new type (array of `int`). This construction can be expressed directly in C as a `typedef`, where the syntactic ordering of the type parameter and return type is different from that of a function but entirely analogous. See Figure 8.19.

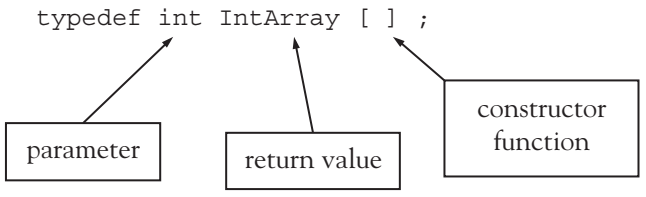

Figure 8.19 The components of a type definition in C

A type declaration in ML using the type constructor `Stack` is essentially the same kind of construction, as shown in Figure 8.20.

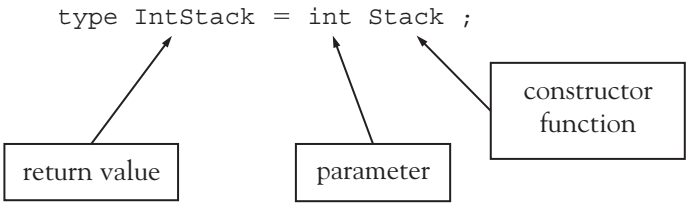

Figure 8.20: The components of a type definition in ML

When comparing an explicitly parameterized polymorphic type construction such as `Stack` in ML to equivalent constructs in C or other languages, we do need to remain aware of the differences among the different uses of the name `Stack` in a declaration like:

```
datatype 'a Stack = EmptyStack
                  | Stack of 'a * ('a Stack);
```

The first occurrence of `Stack` (to the left of the = sign) defines the `Stack` type constructor, while the second occurrence of `Stack` (after the `|`) defines the *data* or *value constructor* `Stack` (the third occurrence of `Stack` is a recursive *use* of the type constructor meaning of `Stack`). Value constructors correspond precisely to the constructors of an object-oriented language; reusing the same name for the value and type constructor is standard practice in a number of languages. In C++ and Java, for example, constructors always have the same name as their associated class (and type). See Chapter 5 for more detail.

C++ is, in fact, an example of a language with explicit parametric polymorphism, but without the associated implicit Hindley-Milner type checking. The mechanism for explicit polymorphism is the **template**, which we introduced in Chapter 5. A template can be used either with functions or the `class` or `struct` type constructor (thus, unlike ML,[23] both functions and data structures have explicit type parameters).

Here, for example, is a version of the `Stack` class introduced in Chapter 5 using an explicit C++ type parameter `T`:[24]

```
template <typename T>
struct StackNode{
   T data;
   StackNode<T> * next;
};
template <typename T>
struct Stack{
   StackNode<T> * theStack;
};
```

[23]Actually, ML functions *can* be written with explicit type parameters (e.g., `ident (x: 'a) = x`), but it is unusual to do so, since Hindley-Milner type checking introduces them automatically.

[24]The keyword `class` was used instead of `typename` in older versions of C++; the new keyword much more clearly denotes the actual meaning of T as a type parameter.

Notice how we must also use a `struct` definition in C++ to get a `Stack` type that is a pointer to `StackNode`—there are no template `typedef`s in C++.

Now we can define an explicitly parameterized `top` function equivalent to the previous ML definition as follows:

```
template <typename T>
T top (Stack<T> s){
   return s.theStack->data;
}
```

We can use these definitions in some sample code as follows:

```
Stack<int> s;
s.theStack = new StackNode<int>;
s.theStack->data = 3;
s.theStack->next = 0;

int t = top(s); // t is now 3
```

Note that the call to `top(s)` does not require that the type parameter of `top` be explicitly given—the type parameter is inferred to be `int` by the compiler from the type of `s`. On the other hand, whenever a `Stack` variable is defined, an actual (nonparameterized) type must be written as the type parameter. Writing a definition such as:

```
Stack s; // illegal C++
```

in C++ is not allowed.

An explicitly parameterized `max` function can also be written in C++ that is equivalent to the implicitly parameterized ML version:

```
template <typename T>
T max (T x, T y, bool (*gt)(T, T)){
   return gt(x,y) ? x : y ;
}
```

To use this function, we can define a function such as:

```
bool gti(int x, int y){
   return x > y;
}
```

and then we can call the `max` function as follows:

```
int larger = max(2,3,gti); // larger is now 3
```

In fact, we can also define a max function that depends implicitly on the existence of a > operator for its type parameter T:

```
template <typename T>
T max(T x, T y){
return x > y ? x : y ;

}
```

We can call this function using values of any type for which the > operator exists:

```
int larger_i = max(2,3); // larger_i is now 3
double larger_d = max(3.1,2.9); // larger_d is now 3.1
Stack<int> s,t,u;
u = max(s,t);
     // Error! operator > not defined for Stack types
```

This form of parametric polymorphism is called **(implicitly) constrained parametric polymorphism**, because it implicitly applies a constraint to the type parameter T—namely, that only a type with the > operator defined for its values can be substituted for the type parameter T. The Java collection classes provide a similar capability, as discussed in Chapter 5. This is a form of parametric polymorphism that ML does not have.[25]

Ada also allows explicitly parameterized functions and, somewhat indirectly, also explicitly parameterized types. The essential difference between Ada and C++ or ML is that Ada allows parameterization only for program **units**, which are separately compiled program segments that can either be a subprogram (i.e., a function or a procedure) or an Ada **package**. An Ada package is essentially a **module**, and in Chapter 11 we discuss modules and their relation to data types. Here we will only briefly sketch how parametric polymorphism can be achieved using Ada units; see Chapter 11 for more detail on Ada units.

An Ada unit that includes parameters is called a **generic unit**. Type parameters must be specified as private, indicating that no specific information is available about which types will be used with the unit. Here is a **package specification** in Ada that implements the same Stack structure example as before, parameterized by a type T:

```
generic
   type T is private;
package Stacks is
   type StackNode;
   type Stack is access StackNode;
   type StackNode is
   record
```

(continues)

[25]A functional language that *does* have implicitly constrained parametric polymorphism is Haskell (see Chapter 4).

(continued)

```
      data: T;
      next: Stack;
   end record;
   function top( s: Stack ) return T ;
end Stacks;
```

Note that the `Stack` data type and the package name are not the same, and that the `top` function is also declared (but not implemented) here, since it is parameterized on the same type `T`. This package specification needs an associated **package body** that provides the actual implementation of the `top` function:

```
package body Stacks is
   function top( s: Stack ) return T is
   begin
     return s.data;
   end top;
end Stacks;
```

Now this Stacks package can be used by a program as follows:

```
with Stacks; -- import the Stacks package
...
package IntStacks is new Stacks(integer);
     -- instantiate the parameterized package
     -- with type integer

use IntStacks;
   -- without this, we must use the package name
   -- before all of its components, e.g.
   -- IntStacks.Stack,
   -- IntStacks.StackNode, IntStacks.pop, etc.

s : Stack := new StackNode;
t : integer;
...
s.data := 3;
s.next := null;
t := top(s); -- t is now 3
```

Explicitly parameterized polymorphic functions can also be defined in Ada using generic units, but since functions are themselves compilation units, a package is not required (unless we wanted to define more than one function at a time in the same place). For example, here is the specification of the `max` function of previous examples in Ada:

```
generic
   type T is private;
   with function gt(x, y : T) return boolean;
function max (x, y : T) return T;
```

Notice that, rather than making the `gt` function a parameter to the `max` function itself, we have included it as a "generic" parameter to the unit; this is **explicitly constrained parametric polymorphism**, which, unlike the implicitly constrained polymorphism of C++, makes explicit that the `max` function depends on the `gt` function, even though it is not in the list of parameters to the function call itself.

The `max` function unit requires also a body, where the actual code for the `max` function is written:

```
-- body of max unit
function max (x,y:T) return T is
begin
if gt(x,y) then return x;
   else return y;
   end if;
end max;
```

Now the `max` function can be used by an Ada program as follows:

```
with max; -- import the max unit
...
function maxint is new max(integer,">");
-- instantiate the max function for integers,
-- using the ">" function as the gt function.

i: integer;
...
i := maxint(1,2); -- i is now 2
```

As is generally true with Ada, everything must be done explicitly in the code. That is, polymorphism must always be explicit, parameterization must make all dependencies explicit, and instantiation must be made explicit. This is true, even to the extent of requiring a new name—e.g., `maxint` above.

8.10 Case Study: Type Checking in TinyAda

In Section 7.8, we added code for static semantic analysis to a parser for TinyAda. In this code, identifiers are checked to ensure that they are declared before they are used and that they are not declared more than once in the same block. Also, the role of an identifier, as a constant, variable, procedure, or type name, is recorded when it is declared. This information is used to ensure that the identifier is being

referenced in an appropriate context. In this section, we develop and partially implement a design for type checking in the parser. As before, this will involve adding new information to the symbol table when an identifier is declared, and looking up that information to enforce semantic restrictions when the identifier is referenced.

8.10.1 Type Compatibility, Type Equivalence, and Type Descriptors

For a statically typed language like TinyAda, the parser must check the type of any identifier when it is referenced. Many of these checks occur when the identifier is an operand in an expression. For example, in the TinyAda expression `X + 1`, the variable `X` must be of type `INTEGER` or a subrange type of `INTEGER`, whereas in the expression `not (A or B)`, the variables `A` and `B` must be of type `BOOLEAN`. The types of identifiers allowed in operands of expressions, thus, must be the types required by the operators. Moreover, because operands can also be any expressions, each expression also must have a type. For example, the subexpression `A or B` in the expression `not (A or B)` must be of type `BOOLEAN`.

Another contextual factor that restricts the types of identifiers and expressions is their placement in assignment statements or as actual parameters to a procedure call. The name on the left side of an assignment statement must be type-compatible with the expression on its right. The expression or name passed as an actual parameter to a procedure call must be type-compatible with the corresponding formal parameter in the procedure's declaration.

Finally, the types of certain elements of declarations are also restricted. For example, the index types of array type definitions must be the ordinal types `BOOLEAN` or `CHAR` or a subrange of `INTEGER`.

TinyAda uses a loose form of name equivalence to determine whether or not two identifiers (or expressions) are type-compatible. In the cases of arrays and enumerations, this means that two identifiers are type-compatible if and only if they were declared using the same type name in their declarations. In the case of the built-in types `INTEGER`, `CHAR`, and `BOOLEAN` and their programmer-defined subrange types, two identifiers are type-compatible if and only if their supertypes are name-equivalent. For example, the supertype of the type name `INDEX` in the following declaration is `INTEGER`:

```
type INDEX is range 1..10;
```

The supertype of `INTEGER` is itself; therefore, a variable of type `INDEX` is type-compatible with a variable of type `INTEGER`, because their supertypes are the same. From these basic facts about type compatibility and type equivalence, the rules for type checking in the TinyAda parser can be derived.

The primary data structure used to represent type attributes is called a **type descriptor**. A type descriptor for a given type is entered into the symbol table when the corresponding type name is introduced. This happens at program startup for the built-in type names `BOOLEAN`, `CHAR`, and `INTEGER`, and later when new type declarations are encountered. When identifier references or expressions are encountered and their types are checked, the parser either compares the type descriptors of two operands or compares a given type descriptor against an expected descriptor. When strict name equivalence is required (for arrays or enumerations), the two type descriptors must be the exact same object. Otherwise (for the built-in ordinal types and their subranges), the two type descriptors must have the exact same supertype descriptor. These rules lead naturally to the implementation of several type-checking methods that appear later in this section.

8.10.2 The Design and Use of Type Descriptor Classes

Conceptually, a type descriptor is like a variant record, which contains different attributes depending on the category of data type being described. Here is the information contained in the type descriptors for each category of data type in TinyAda.

1. Each type descriptor includes a field called a **type form**, whose possible values are `ARRAY`, `ENUM`, `SUBRANGE`, and `NONE`. This field serves to identify the category of the data type and allows the user to access its specific attributes.
2. The array type descriptor includes attributes for the index types and the element type. These attributes are also type descriptors.
3. The enumeration type descriptor includes a list of symbol entries for the enumerated constant names of that type.
4. The type descriptors for subrange types (including `BOOLEAN`, `CHAR`, and `INTEGER`) include the values of the lower bound and upper bound, as well as a type descriptor for the supertype.

There is no variant record structure in Java to represent this information, but we can model it nicely with a `TypeDescriptor` class and three subclasses, `ArrayDescriptor`, `SubrangeDescriptor`, and `EnumDescriptor`. The top-level class includes the type form, which is automatically set when an instance of any subclass is created. The subclasses include the information just listed, with the representation of records and enumerated types merged into one class. Each class also includes methods to set its attributes. The user can access these directly as public fields. The complete implementation of these classes is available on the book's Web site.

At program startup, the parser now creates type descriptors for the three built-in types. These descriptors are declared as instance variables of the parser, for easy access and because they will be the unique supertypes of all subrange types. We define a new method to initialize these objects and update another method to attach them as types to the built-in identifiers of TinyAda. Here is the code for these changes:

```
// Built-in type descriptors, globally accessible within the parser
private SubrangeDescriptor BOOL_TYPE, CHAR_TYPE, INT_TYPE;

// Sets up the type information for the built-in type descriptors
private void initTypeDescriptors(){
   BOOL_TYPE = new SubrangeDescriptor();
   BOOL_TYPE.setLowerAndUpper(0, 1);
   BOOL_TYPE.setSuperType(BOOL_TYPE);
   CHAR_TYPE = new SubrangeDescriptor();
   CHAR_TYPE.setLowerAndUpper(0, 127);
   CHAR_TYPE.setSuperType(CHAR_TYPE);
   INT_TYPE = new SubrangeDescriptor();
   INT_TYPE.setLowerAndUpper(Integer.MIN_VALUE, Integer.MAX_VALUE);
```

(continues)

(continued)

```
      INT_TYPE.setSuperType(INT_TYPE);
   }

   // Attach type information to the built-in identifiers
   private void initTable(){
      table = new SymbolTable(chario);
      table.enterScope();
      SymbolEntry entry = table.enterSymbol("BOOLEAN");
      entry.setRole(SymbolEntry.TYPE);
      entry.setType(BOOL_TYPE);
      entry = table.enterSymbol("CHAR");
      entry.setRole(SymbolEntry.TYPE);
      entry.setType(CHAR_TYPE);
      entry = table.enterSymbol("INTEGER");
      entry.setRole(SymbolEntry.TYPE);
      entry.setType(INT_TYPE);
      entry = table.enterSymbol("TRUE");
      entry.setRole(SymbolEntry.CONST);
      entry.setType(BOOL_TYPE);
      entry = table.enterSymbol("FALSE");
      entry.setRole(SymbolEntry.CONST);
      entry.setType(BOOL_TYPE);
   }
```

The parser also defines two utility methods to check types. The first, `matchTypes`, compares two type descriptors for compatibility using name equivalence. The second, `acceptTypeForm`, compares a type descriptor's type form to an expected type form. Each method outputs an error message if a type error occurs, as shown in the following code:

```
   // Compares two types for compatibility using name equivalence
   private void matchTypes(TypeDescriptor t1, TypeDescriptor t2,
                           String errorMessage){
      // Need two valid type descriptors to check for an error
      if (t1.form == TypeDescriptor.NONE || t2.form == TypeDescriptor.NONE)
       return;
      // Exact name equivalence: the two descriptors are identical
      if (t1 == t2) return;
      // To check two subranges, the supertypes must be identical
      if (t1.form == TypeDescriptor.SUBRANGE && t2.form == TypeDescriptor.
       SUBRANGE){
         if (((SubrangeDescriptor)t1).superType != ((SubrangeDescriptor)t2).
          superType)
            chario.putError(errorMessage);
      }
      else
```

(continues)

(continued)

```
        // Otherwise, at least one record, array, or enumeration does not match
        chario.putError(errorMessage);
  }

  // Checks the form of a type against an expected form
  private void acceptTypeForm(TypeDescriptor t, int typeForm, String
   errorMessage){
     if (t.form == TypeDescriptor.NONE) return;
     if (t.form != typeForm)
        chario.putError(errorMessage);
  }
```

Note that the formal type descriptor parameters of these two methods are declared to be of the class `TypeDescriptor`, which allows the methods to receive objects of any of the type descriptor subclasses as actual parameters. Both methods then check type forms to determine which actions to take. If the form is `NONE`, a semantic error has already occurred somewhere else, so the methods quit to prevent a ripple effect of error messages. Otherwise, further checking occurs. In the case of `matchTypes`, Java's == operator is used to detect the object identity of the two type descriptors. This can be done without knowing which classes these objects actually belong to. However, if the two descriptors are not identical, the types will be compatible only if they are both subranges and their supertypes are identical. After determining that the form of the two descriptors is `SUBRANGE`, the `TypeDescriptor` objects must be cast down to the `SubrangeDescriptor` class before accessing their supertype fields. Fortunately, this type of downcasting will rarely occur again in the parser; the use of `TypeDescriptor` as a parameter type and a method return type will provide very clean, high-level communication channels between the parsing methods.

8.10.3 Entering Type Information in Declarations

Type information must now be entered wherever identifiers are declared in a source program. We noted these locations in Section 8.8, where the parser enters identifier role information. The type information comes either from type identifiers (in number or object declarations, parameter specifications, and record component declarations) or from a type definition. In the case of a type identifier, the type descriptor is already available and is simply accessed in the identifier's symbol entry. In the case of a type definition, a new type might be created. Therefore, the `typeDefinition` method should return to its caller the `TypeDescriptor` for the type just processed. The following code shows the implementation of this method and its use in the method `TypeDeclaration`, which declares a new type name:

```
  /*
  typeDeclaration = "type" identifier "is" typeDefinition ";"
  */
  private void typeDeclaration(){
     accept(Token.TYPE, "'type' expected");
     SymbolEntry entry = enterId();
     entry.setRole(SymbolEntry.TYPE);
```

(continues)

(continued)

```
      accept(Token.IS, "'is' expected");
      entry.setType(typeDefinition());            // Pick up the type
       descriptor here
      accept(Token.SEMI, "semicolon expected");
   }

   /*
   typeDefinition = enumerationTypeDefinition | arrayTypeDefinition
                  | range | <type>name
   */
   private TypeDescriptor typeDefinition(){
      TypeDescriptor t = new TypeDescriptor();
      switch (token.code){
         case Token.ARRAY:
            t = arrayTypeDefinition();
            break;
         case Token.L_PAR:
            t = enumerationTypeDefinition();
            break;
         case Token.RANGE:
            t = range();
            break;
         case Token.ID:
            SymbolEntry entry = findId();
            acceptRole(entry, SymbolEntry.TYPE, "type name expected");
            t = entry.type;
            break;
         default: fatalError("error in type definition");
      }
      return t;
   }
```

Note how each lower-level parsing method called in the `typeDefinition` method also returns a type descriptor, which is essentially like a buck being passed back up the call tree of the parsing process. We see the buck stopping at one point, when the type definition is just another identifier, whose type descriptor is accessed directly. The strategy of passing a type descriptor up from lower-level parsing methods is also used in processing expressions, as we will see shortly.

The next example shows how a new type is actually created. An enumeration type consists of a new set of constant identifiers. Here is the code for the method `enumerationTypeDefinition`:

```
   /*
   enumerationTypeDefinition = "(" identifierList ")"
   */
   private TypeDescriptor enumerationTypeDefinition(){
      accept(Token.L_PAR, "'(' expected");
```

(continues)

(continued)

```
      SymbolEntry list = identifierList();
      list.setRole(SymbolEntry.CONST);
      EnumDescriptor t = new EnumDescriptor();
      t.setIdentifiers(list);
      list.setType(t);
      accept(Token.R_PAR, "')' expected");
      return t;
   }
```

The method now returns a type descriptor. The three new lines of code in the middle of the method

- Instantiate a type descriptor for an enumerated type.
- Set its `identifiers` attribute to the list of identifiers just parsed.
- Set the type of these identifiers to the type descriptor just created.

The processing of array and subrange type definitions is similar and left as an exercise for the reader.

8.10.4 Checking Types of Operands in Expressions

A quick glance at the rules for TinyAda expressions gives some hints as to how their types should be checked. Working top-down, a condition must be a Boolean expression, so clearly the `expression` method must return a type descriptor to be checked. Here is the new code for the `condition` method:

```
   /*
   condition = <boolean>expression
   */
   private void condition(){
      TypeDescriptor t = expression();
      matchTypes(t, BOOL_TYPE, "Boolean expression expected");
   }
```

The `expression` method in turn might see two or more relations separated by Boolean operators. Thus, the `relation` method must also return a type descriptor. If a Boolean operator is detected in the `expression` method, then the types of the two relations must each be matched against the parser's `BOOL_TYPE` variable.

The coder of the parser must read each syntax rule for expressions carefully and add the appropriate code for type checking to the corresponding method. As an example, here is the code for the method `factor`, which shows the checking of some operands as well as the return of the appropriate type descriptor:

```
/*
factor = primary [ "**" primary ] | "not" primary
*/
private TypeDescriptor factor(){
   TypeDescriptor t1;
   if (token.code == Token.NOT){
      token = scanner.nextToken();
      t1 = primary();
      matchTypes(t1, BOOL_TYPE, "Boolean expression expected");
   }
   else{
      t1 = primary();
      if (token.code == Token.EXPO){
         matchTypes(t1, INT_TYPE, "integer expression expected");
         token = scanner.nextToken();
         TypeDescriptor t2 = primary();
         matchTypes(t2, INT_TYPE, "integer expression expected");
      }
   }
   return t1;
}
```

This method has the following three options for returning a type descriptor:

1. There is no operator in the factor, so the type descriptor `t1` obtained from the first call of `primary` is returned directly to the caller.
2. The operator is a leading `not`. Then the type descriptor `t1` is returned, after matching it to `BOOL_TYPE`.
3. The operator is a binary `**`. Then the type descriptor `t1` is returned, after matching it and the type descriptor `t2`, obtained from the second call to `primary`, to `INT_TYPE`.

As you can see, the type of every operand must be checked, and great care must be taken to ensure that the right type descriptor is returned. The modification of the remaining methods for parsing expressions is left as an exercise for the reader.

8.10.5 Processing Names: Indexed Component References and Procedure Calls

The syntax of TinyAda indexed component references and procedure calls is the same if the procedure expects at least one parameter. As shown in the previous code segment, the method `name` now must distinguish these two types of phrases, based on the role of the leading identifier. The code for processing the trailing phrase, which consists of a parenthesized list of expressions, can now be split into two distinct parsing methods, `actualParameterPart` and `indexedComponent`.

The method `actualParameterPart` receives the procedure name's symbol entry as a parameter. The entry contains the information for the procedure's formal parameters, which was entered earlier when the procedure was declared. As it loops through the list of expressions, this method should match the type of each expression to the type of the formal parameter in that position. Also, the number of expressions and formal parameters must be the same.

The method `indexedComponent` receives the array identifier's entry as a parameter. This entry's type descriptor contains the information for the array's index types and element type, which was entered earlier when that array type was defined. This method uses the same strategy to match expressions to declared index types as we saw for matching the types of formal and actual parameters in the method `actualParameterPart`. However, `indexedComponent` must also return a new symbol entry that contains the array's element type, as well as the role of the array identifier. This will be the information returned by the `name` method for role and type analysis farther up the call tree. The completion of these two methods, as well as the rest of the code for type analysis in the TinyAda parser, is left as an exercise for the reader.

8.10.6 Completing Static Semantic Analysis

Two other types of semantic restrictions can be imposed during parsing. One restriction involves the checking of **parameter modes**. TinyAda has three parameter modes: input only (signified by the keyword `in`), output only (signified by the keyword `out`), and input/output (signified by the keywords `in out`). These modes are used to control the flow of information to and from procedures. For example, only a variable or a parameter with the same mode can be passed as an actual parameter where the formal parameter has been declared as `out` or `in out`. Within a procedure's code, an `out` parameter cannot appear in an expression, and an `in` parameter cannot be the target of an assignment statement.

The other restriction recognizes a distinction between static expressions and other expressions. A static expression is one whose value can be computed at compile time. Only static expressions can be used in number declarations and range type definitions. We will discuss the enforcement of these two types of restrictions in later chapters.

Exercises

8.1 Compare the flexibility of the dynamic typing of Lisp with the reliability of the static typing of Ada, ML, C++, or Java. Think of as many arguments as you can both for and against static typing as an aid to program design.

8.2 Look up the implementation details on the representation of data by your C/C++ compiler (or other non-Java compiler) that you are using and compare them to the sketches of typical implementations given in the text. Where do they differ? Where are they the same?

8.3 **(a)** The Boolean data type may or may not have an order relation, and it may or may not be convertible to integral types, depending on the language. Compare the treatment of the Boolean type in at least two of the following languages with regard to these two issues: Java, C++, Ada, ML.

(b) Is it useful for Booleans to be ordered? Convertible to integers?

8.4 **(a)** Given the C declarations

```
struct{
   int i;
   double j;
} x, y;
struct{
   int i;
   double j;
} z;
```

the assignment `x = z` generates a compilation error, but the assignment `x = y` does not. Why?

(b) Give two different ways to fix the code in part (a) so that `x = z` works. Which way is better and why?

8.5 Suppose that we have two C arrays:

```
int x[10];
int y[10];
```

Why won't the assignment `x = y` compile in C? Can the declarations be fixed so the assignment will work?

8.6 Given the following variable declarations in C/C++:

```
enum { one } x;
enum { two } y;
```

the assignment `x = y` is fine in C, but generates a compiler error in C++. Explain.

8.7 Given the following function variable declarations in C/C++:

```
int (*f)(int);
int (*g)(int);
int (*h)(char);
```

the assignment `f = g` is fine in C and C++, but the assignment `h = g` generates a compiler warning in C and a compiler error in C++. Explain.

8.8 Show that the set:

```
{emptylist} ∪ char ∪ (char × char) ∪
(char × char × char) ∪...
```

is the smallest solution to the set equation:

```
CharList = {emptylist} ∪ char × CharList
```

(See Section 8.3.5 and Exercise 6.53.)

8.9 Consider the set equation

$$\chi \times \text{char} = \chi$$

(a) Show that any set X satisfying this equation must be infinite.
(b) Show that any element of a set satisfying this equation must contain an infinite number of characters.
(c) Is there a smallest solution to this equation?

8.10 Consider the following declaration in C syntax:

```
struct CharTree{
   char data;
   struct CharTree left, right;
};
```

(a) Rewrite `CharTree` as a union, similar to the `union CharList` declaration of Section 8.3.5.
(b) Write a recursive set equation for your declaration in part (a).
(c) Describe a set that is the smallest solution to your equation of part (b).
(d) Prove that your set in part (c) is the smallest solution.
(e) Rewrite this as a valid C declaration.

8.11 Here are some type declarations in C:

```
typedef struct Rec1 * Ptr1;
typedef struct Rec2 * Ptr2;
struct Rec1{
int data;
   Ptr2 next;
};
struct Rec2{
   double data;
   Ptr2 next;
};
```

Should these be allowed? Are they allowed? Why or why not?

8.12 **(a)** What is the difference between a subtype and a derived type in Ada?
(b) What C type declaration is the following Ada declaration equivalent to?

```
subtype New_Int is integer;
```

(c) What C type declaration is the following Ada declaration equivalent to?

```
type New_Int is new integer;
```

8.13 In Ada the equality test (`x = y` in Ada syntax) is legal for all types, as long as `x` and `y` have the same type. In C, however, the equality test (`x == y` in C syntax) is only permitted for variables of a few types; however, the types of `x` and `y` need not always be the same.
(a) Describe the types of `x` and `y` for which the C comparison `x == y` is legal.
(b) Why does the C language not allow equality tests for variables of certain types?
(c) Why does the C language allow comparisons between certain values of different types?

8.14 Describe the type correctness and inference rules in C for the conditional expression:

```
e1 ? e2 : e3
```

Must `e2` and `e3` have the same type? If they have the same type, can it be any type?

8.15 Suppose we used the following C++ code that attempts to avoid the use of a `static_cast` to print out the internal value of `true` (see the code at the end of Section 8.7):

```
union{
   int i;
   bool b;
} x;
...
x.b = true;
cout << x.i << endl;
```

Why is this wrong? (*Hint:* Try the assignment `x.i=20000` before `x.b=true.`)

8.16 Programmers often forget that numeric data types such as `int` are only approximations of mathematical number systems. For example, try the following two divisions in C or C++ or Java on your system, and explain the result: `-2147483648 / -1; -2147483647 / -1.`

8.17 The chapter makes no mention of the initialization problem for variables. Describe the rules for variable initialization in C, Ada, ML, or other similar language. In particular, are variables of all types initialized when a program starts running? If so, how is the initial value constructed?

8.18 Here are some type and variable declarations in C syntax:

```
typedef char* Table1;
typedef char* Table2;

Table1 x,y;
Table2 z;
Table2 w;
```

State which variables are type equivalent under **(a)** structural equivalence, **(b)** name equivalence, and **(c)** the actual C equivalence algorithm. Be sure to identify those cases that are ambiguous from the information at hand.

8.19 Given the declarations

```
int x[10];
int y[5];
```

are `x` and `y` type equivalent in C?

8.20 Given the following C declaration:

```
const PI = 3.14159;
```

if we then use the constant `PI` in a geometry formula (like `A = PI*r*r`), we get the wrong answer. Why?

8.21 Given the following Java declaration:

```
short i = 2;
```

the Java compiler generates an error for the statement:

```
i = i + i;
```

Why?

8.22 Here are some type and variable declarations in C syntax:

```
typedef struct{
   int x;
   char y;
} Rec1;
typedef Rec1 Rec2;
typedef struct{
   int x;
   char y;
} Rec3;
Rec1 a,b;
Rec2 c;
Rec3 d;
```

State which variables are type equivalent under **(a)** structural equivalence, **(b)** name equivalence, and **(c)** the actual C equivalence algorithm. Be sure to identify those cases that are ambiguous from the information at hand.

8.23 In C++ the equality test == can be applied to arrays, but it tests the wrong thing. Also, the assignment operator = cannot be applied to arrays at all. Explain.

8.24 Can equality and assignment be applied to Ada arrays? Do they test the right thing?

8.25 By the type rules stated in the text, the array constructor in C does not construct a new type; that is, structural equivalence is applied to arrays. How could you write code to test this? (*Hint:* Can we use an equality test? Are there any other ways to test compatibility?)

8.26 Can a union in C be used to convert `int`s to `double`s and vice versa? To Convert `long`s to `float`s? Why or why not?

8.27 In a language in which "/" can mean either integer or real division and that allows coercions between integers and reals, the expression `I + J / K` can be interpreted in two different ways. Describe how this can happen.

8.28 Show how Hindley-Milner type checking determines that the following function (in ML syntax) takes an integer parameter and returns an integer result (i.e., is of type `int -> int` in ML terminology):

```
fun fac n = if n = 0 then 1 else n * fac (n-1);
```

8.29 Here is a function in ML syntax:

```
fun f (g,x) = g (g(x));
```

(a) What polymorphic type does this function have?
(b) Show how Hindley-Milner type checking infers the type of this function.
(c) Rewrite this function as a C++ template function.

8.30 Write a polymorphic `swap` function that swaps the values of two variables of the same type in:
(a) C++
(b) ML

8.31 Write a polymorphic `max` function in C++ that takes an array of values of any type and returns the maximum value in the array, assuming the existence of a ">" operation.

8.32 Repeat the previous exercise, but add a parameter for a `gt` function that takes the place of the ">" operation.

8.33 Can both functions of the previous two exercises exist (and be used) simultaneously in a C++ program? Explain.

8.34 Explicit parametric polymorphism need not be restricted to a single type parameter. For example, in C++ one can write:

```
template <typename First, typename Second>
struct Pair{
   First first;
   Second second;
};
```

Write a C++ template function `makePair` that takes two parameters of different types and returns a `Pair` containing its two values.

8.35 Write the `Pair` data structure of the previous exercise as an Ada generic package, and write a similar `makePair` function in Ada.

8.36 Is C++ parametric polymorphism let-bound? (*Hint:* See the functions `makepair` and `makepair2` near the end of Section 8.8.

8.37 Does the occur-check problem occur in C++? Explain.

8.38 A problem related to let-bound polymorphism in ML is that of **polymorphic recursion**, where a function calls itself recursively, but on different types at different points in the code. For example, consider the following function (in ML syntax):

```
fun f x = if x = x then true else (f false);
```

What type does this function have in ML? Is this the most general type it could have? Show how ML arrived at its answer.

8.39 Does the polymorphic recursion problem of the previous exercise exist in C++? Explain.

8.40 Data types can be kept as part of the syntax tree of a program and checked for structural equivalence by a simple recursive algorithm. For example, the type

```
struct{
   double x;
   int y[10];
};
```

might be kept as the tree shown in Figure 8.21.

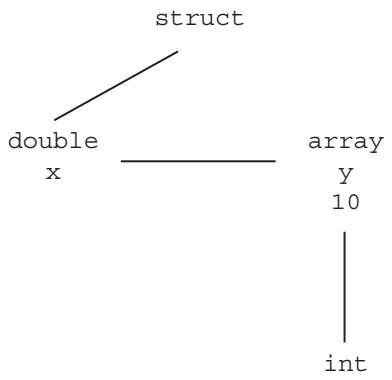

Figure 8.21: A syntax tree for the struct definition

Describe a tree node structure that could be used to express the types of C or a similar language as trees. Write out a `TypeEqual` function in pseudocode that would check structural equivalence on these trees.

8.41 How could the trees of the previous exercise represent recursive types? Modify your `TypeEqual` function to take recursive types into account.

8.42 The language C uses structural type equivalence for arrays and pointers, but declaration equivalence for structs and unions. Why do you think the language designers did this?

8.43 Compare the BNFs for type declarations in C (Java, Ada) with the type tree of Figure 8.5 (8.6, 8.7). To what extent is the tree a direct reflection of the syntax?

8.44 What new type constructors does C++ add to those of C? Add these to the type tree of Figure 8.5. (*Hint:* Use the BNFs for C++ as motivation.)

8.45 The type tree for Ada (Figure 8.7) is somewhat simplified, with a number of constructors omitted. Add the missing constructors. Are there any missing simple types?

8.46 What operations would you consider necessary for a string data type? How well do arrays of characters support these operations?

8.47 Given the following C++ declarations,

```
double* p = new double(2);
void* q;
int* r;
```

which of the following assignments does the compiler complain about?

```
q = p;
p = q;
r = p;
p = r;
r = q;
r = (int*)q;
r = static_cast<int*>(q);
r = static_cast<int*>(p);
r = reinterpret_cast<int*>(p);
```

Try to explain the behavior of the compiler. Will `*r` ever have the value 2 after one of the assignments to `r`? Why?

8.48 In Java, the following local variable declaration results in a compiler error:

```
float x = 2.1;
```

Why? Find two ways of removing this error.

8.49 Should the equality test be available for floating-point types (explain)? Is it available in Java? Ada? ML?

8.50 Should the equality test be available for function types (explain)? Is it available in C? Ada? ML?

8.51 The conversion rules for an arithmetic expression such as $e1 + e2$ in C are stated in Kernighan and Ritchie [1978] as follows (these are the pre-ANSI rules, which are simpler than the ANSI rules, but have the same result in many cases):

> First, any operands of type `char` or `short` are converted to `int`, and any of type `float` are converted to `double`. Then, if either operand is `double`, the other is converted to `double` and that is the type of the result.
>
> Otherwise, if either operand is `long`, the other is converted to `long` and that is the type of the result.
>
> Otherwise, if either operand is `unsigned`, the other is converted to `unsigned` and that is the type of the result.
>
> Otherwise, both operands must be `int`, and that is the type of the result.

Given the following expression in C, assuming `x` is an `unsigned` with value 1, describe the type conversions that occur during evaluation and the type of the resulting value. What is the resulting value?

```
'0' + 1.0 * (-1 + x)
```

8.52 In C there is no Boolean data type. Instead, comparisons such as `a == b` and `a <= b` return the integer value 0 for false and 1 for true. On the other hand, the `if` statement in C considers any nonzero value of its condition to be equivalent to true. Is there an advantage to doing this? Why not allow the condition `a <= b` to return any nonzero value if it is true?

8.53 Complete the entry of type information for the TinyAda parser discussed in Section 8.10. You should modify the type definition methods and the parameter declaration methods. Be sure to add type checking to these methods where relevant. Note that the `SymbolEntry` class and the `TypeDescriptor` class include two methods for setting formal parameter lists and lists of array index types, respectively. Test your parser with example TinyAda programs that exercise all of the modified methods.

8.54 Complete the modifications for type analysis to the TinyAda parsing methods that process expressions. Be sure to add type checking to these methods where relevant. Test your parser with example TinyAda programs that exercise all of the modified methods.

8.55 Complete the modifications for type analysis to the TinyAda parsing methods that process indexed variable references. Note that the `ArrayDescriptor` class includes the method `getIndexes`, which returns an iterator on the array's index types.

8.56 Complete the modifications for type analysis to the TinyAda parsing methods that process actual parameters of procedure calls. Note that the `SymbolEntry` class includes the method `getParameters`, which returns an iterator on the procedure's formal parameters.

8.57 Complete the modifications for type analysis to the TinyAda parsing methods that process names and assignment or procedure call statements.

Notes and References

Data type systems in Algol-like languages such as Pascal, C, and Ada seem to suffer from excessive complexity and many special cases. In part, this is because a type system is trying to balance two opposing design goals: expressiveness and security. That is, a type system tries to make it possible to catch as many errors as possible while at the same time allowing the programmer the flexibility to create and use as many types as are necessary for the natural expression of an algorithm. ML (as well as Haskell) escapes from this complexity somewhat by using structural equivalence for predefined structures but name equivalence for user-defined types; however, this does weaken the effectiveness of type names in distinguishing different uses of the same structure. An interesting variant on this same idea is Modula-3, an object-oriented modification of Modula-2 that uses structural equivalence for ordinary data and name equivalence for objects (Cardelli et al. [1989a,b]; Nelson [1991]). Ada has perhaps the most consistent system, albeit containing many different ideas and hence extremely complex.

One frustrating aspect of the study of type systems is that language reference manuals rarely state the underlying type-checking algorithms explicitly. Instead, the algorithms must be inferred from a long list of special rules, a time-consuming investigative task. There are also few references that give detailed overviews of language type systems. Two that do have considerable detail are Cleaveland [1986] and Bishop [1986]. A lucid description of Pascal's type system can be found in Cooper [1983]. A very detailed description of Ada's type system can be found in Cohen [1996]. Java's type system is described in Gosling et al. [2000], C's type system in Kernighan and Ritchie [1988], C++'s type system

in Stroustrup [1997], and ML's type system in Ullman [1998] and Harper and Mitchell [1993]; Haskell's in Thompson [1999].

For a study of type polymorphism and overloading, see Cardelli and Wegner [1985]. The Hindley-Milner algorithm is clearly presented in Cardelli [1987], together with Modula-2 code that implements it for a small sample language; let-bound polymorphism and the occur-check are also discussed. Let-bound polymorphism is also described in Ullman [1998], Section 5.3, where "generic" type variables are called "generalizable." C++ templates are amply described in Stroustrup [1997]. Ada "generics" are described in Cohen [1996]. Polymorphic recursion (Exercise 9.38) is described in Henglein [1993].

The IEEE floating-point standard, mentioned in Section 8.2, appears in IEEE [1985].